

**VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY**



**Digital System
Lab 1**

Instructor: Đoàn Ngọc Cẩm

Student	Student ID
Hồ Chí Dũng	2351016
Nguyễn Viết Thi	2451165
Trần Duy Anh	2351007

Ho Chi Minh City – February 2026

Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

OBJECTIVES

- The purpose of this lab is to learn how to connect simple input (switches) and output devices (LEDs and 7-segment) to an FPGA chip and implement a circuit that uses these devices.
- Examine arithmetic circuits that add, subtract, and multiply numbers.

PREPARATION FOR LAB 1

- Finish Pre Lab 1 at home.
- Students have to simulate all the exercises in Pre Lab 1 at home. All results (codes, waveform, RTL viewer, ...) have to be captured and submitted to instructors prior to the lab session.

If not, students will not participate in the lab and be considered absent this session.

REFERENCE

1. Intel FPGA training

EXPERIMENT 1

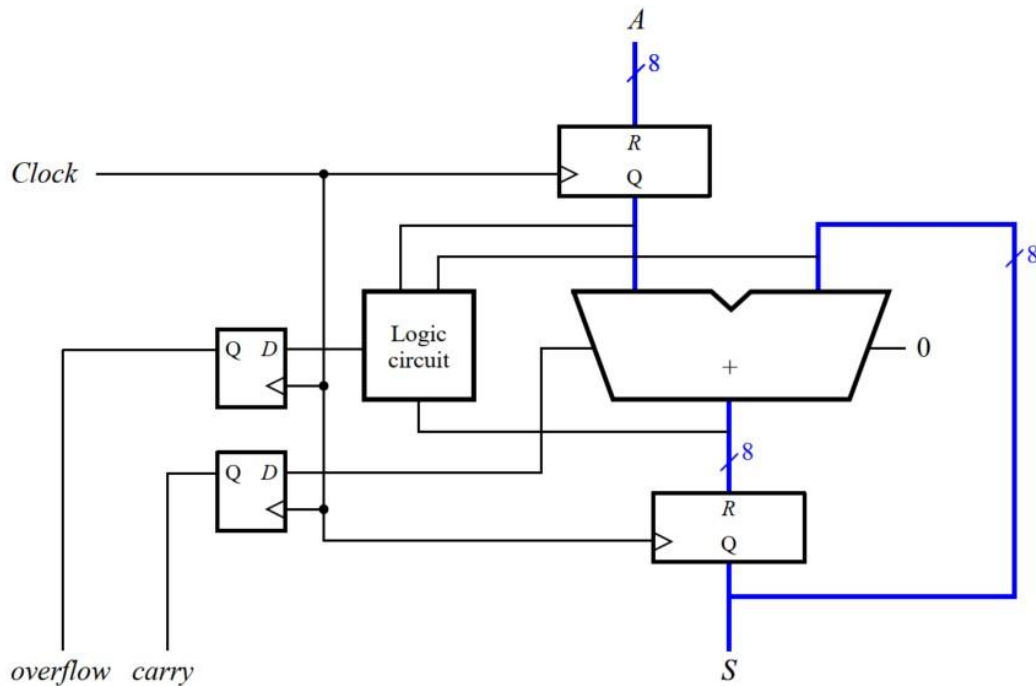
Objective: Known how to program a system to add the value of an input A to itself repeatedly.

Requirement: The circuit in Figure 1, which is often called an *accumulator*, is used to add the value of an input A to itself repeatedly. The circuit includes a carry out from the adder, as well as an *overflow* output signal. If the input A is considered as a 2's-complement number, then *overflow* should be set to 1 in the case where the output *sum* produced does not represent a correct 2's complement result.



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Figure

L: An eight-bit accumulator circuit.

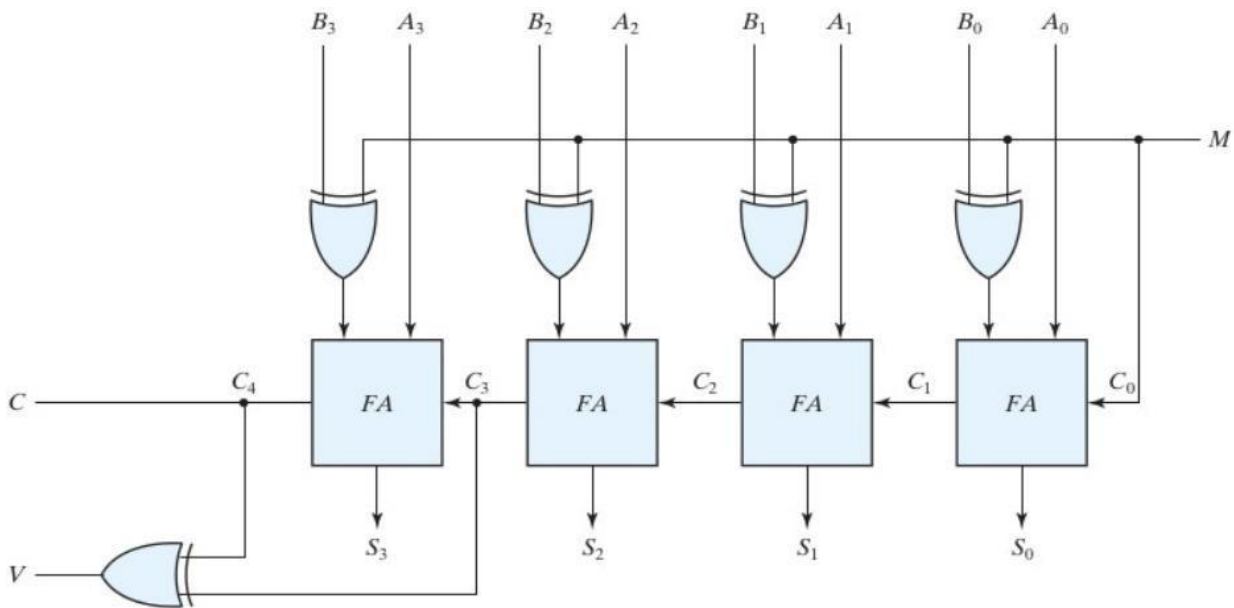
Instruction:

- Write System Verilog code to describe 4-bit adder circuit.
- There are two ways to detect overflow for 2's complement:
 - Compare inputs and outputs: The inputs and outputs always have the same sign. If not, the overflow must be set.
 - Compare carry bits: The circuit for checking overflow of 4-bit adder is shown below.



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



In the figure, V is the overflow flag.

- Write System Verilog code to describe the circuit given in Figure 1.
- Connect input A to switches SW7–0, use KEY0 as an active-low asynchronous reset, and use KEY1 as a manual clock input. The sum from the adder should be displayed on the red lights LEDR7–0, the registered carry signal should be displayed on LEDR8, and the registered overflow signal should be displayed on LEDR9. Show the registered values of A and S as hexadecimal numbers on the 7-segment displays HEX3–2 and HEX1 – 0.

Check: Your report has to show two results:

- The waveform to prove the circuit works correctly. You must show all the cases of overflow flag and carry flag.
- The result of RTL viewer.



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Code

```
//Module Giải mã 7-segment
module hex_7seg (
    input logic [3:0] hex,
    output logic [6:0] seg
);
    // Bảng chân trị cho bộ giải mã 7 đoạn (Active-low cho FPGA Intel)
    always_comb begin
        case (hex)
            4'h0: seg = 7'b1000000; 4'h1: seg = 7'b1111001;
            4'h2: seg = 7'b0100100; 4'h3: seg = 7'b0110000;
            4'h4: seg = 7'b0011001; 4'h5: seg = 7'b0010010;
            4'h6: seg = 7'b0000010; 4'h7: seg = 7'b1111000;
            4'h8: seg = 7'b0000000; 4'h9: seg = 7'b0010000;
            4'hA: seg = 7'b0001000; 4'hB: seg = 7'b0000011;
            4'hC: seg = 7'b1000110; 4'hD: seg = 7'b0100001;
            4'hE: seg = 7'b0000110; 4'hF: seg = 7'b0001110;
            default: seg = 7'b1111111;
        endcase
    end
endmodule

//Khoi D-Flip Flop
module dff_en (
    input logic d, clk, rst_n, en,
    output logic q
);
    // Reset mức thấp (negedge) theo yêu cầu đề bài
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            q <= 1'b0;
        else if (en)
            q <= d;
    end
endmodule

// Thanh ghi 8-bit
module reg_8bit_struct (
    input logic [7:0] D,
    input logic clk, rst_n, en,
    output logic [7:0] Q
);
    // Lắp ghép từng bit một, không dùng vòng lặp for
    dff_en bit0 (D[0], clk, rst_n, en, Q[0]);
    dff_en bit1 (D[1], clk, rst_n, en, Q[1]);
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
dff_en bit2 (D[2], clk, rst_n, en, Q[2]);
dff_en bit3 (D[3], clk, rst_n, en, Q[3]);
dff_en bit4 (D[4], clk, rst_n, en, Q[4]);
dff_en bit5 (D[5], clk, rst_n, en, Q[5]);
dff_en bit6 (D[6], clk, rst_n, en, Q[6]);
dff_en bit7 (D[7], clk, rst_n, en, Q[7]);
endmodule

// Full Adder
module full_adder (
    input logic a, b, cin,
    output logic s, cout
);
    assign s = a ^ b ^ cin;
    assign cout = (a & b) | (b & cin) | (a & cin);
endmodule

// Module bộ cộng 8-bit sử dụng cấu trúc Ripple Carry
module adder_8bit (
    input logic [7:0] a, b,
    input logic cin,
    output logic [7:0] sum,
    output logic cout,
    output logic c7_out
);
    logic [7:1] c; // Các đường mang nhớ trung gian

    // Kết nối các bộ cộng toàn phần (Full Adders)
    full_adder fa0 (a[0], b[0], cin, sum[0], c[1]);
    full_adder fa1 (a[1], b[1], c[1], sum[1], c[2]);
    full_adder fa2 (a[2], b[2], c[2], sum[2], c[3]);
    full_adder fa3 (a[3], b[3], c[3], sum[3], c[4]);
    full_adder fa4 (a[4], b[4], c[4], sum[4], c[5]);
    full_adder fa5 (a[5], b[5], c[5], sum[5], c[6]);
    full_adder fa6 (a[6], b[6], c[6], sum[6], c[7]);
    full_adder fa7 (a[7], b[7], c[7], sum[7], cout);
    assign c7_out = c[7];
endmodule

//khởi logic phát hiện tràn
module overflow_det (
    input logic c7,
    input logic c8,
    output logic v
);
    // Tràn xảy ra khi mang nhớ vào bit đầu khác với mang nhớ ra khỏi bit đầu
    assign v = c7 ^ c8;
endmodule
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
module lab1_tn1 (
    input logic [7:0] SW,          // SW7-0: Input A [cite: 90]
    input logic [1:0] KEY,        // KEY0: Reset_n, KEY1: Clock [cite: 90]
    output logic [9:0] LEDR,      // LEDR7-0: S, LEDR8: Carry, LEDR9: Overflow
    output logic [6:0] HEX0, HEX1, HEX2, HEX3 // Hiển thị A và S
);
    // Tín hiệu nội bộ
    logic [7:0] A_reg, S_accumulator, next_sum;
    logic c7_sig, c8_sig, v_sig;

    // 1. Khối thanh ghi đầu vào A (Register R trong Figure 1)
    // Sử dụng KEY1 làm Clock và KEY0 làm Reset không đồng bộ
    reg_8bit_struct regA (
        .D(SW),
        .clk(KEY[1]),
        .rst_n(KEY[0]),
        .en(1'b1),
        .Q(A_reg)
    );

    // 2. Bộ cộng 8-bit (Accumulator logic: S = S + A) [cite: 19]
    adder_8bit adder_inst (
        .a(A_reg),
        .b(S_accumulator),
        .cin(1'b0),
        .sum(next_sum),
        .cout(c8_sig),
        .c7_out(c7_sig)
    );

    // 3. Khối kiểm tra lỗi Tràn (Overflow: V = C7 ^ C8)
    assign v_sig = c7_sig ^ c8_sig;

    // 4. Thanh ghi lưu trữ Tổng (Accumulator Register S) [cite: 29]
    reg_8bit_struct regS (
        .D(next_sum),
        .clk(KEY[1]),
        .rst_n(KEY[0]),
        .en(1'b1),
        .Q(S_accumulator)
    );

    // 5. Chốt tín hiệu Carry và Overflow vào thanh ghi (Registered Flags)
    dff_en carry_ff (c8_sig, KEY[1], KEY[0], 1'b1, LEDR[8]);
    dff_en overflow_ff (v_sig, KEY[1], KEY[0], 1'b1, LEDR[9]);

    // Hiển thị Tổng S lên đèn LEDR7-0
    assign LEDR[7:0] = S_accumulator;
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
// 6. Hiển thị Hexadecimal lên các bộ 7-segment
// HEX3-2 hiển thị giá trị thanh ghi A (A_reg)
hex_7seg h3(A_reg[7:4], HEX3);
hex_7seg h2(A_reg[3:0], HEX2);
// HEX1-0 hiển thị giá trị tổng hiện tại (S_accumulator)
hex_7seg h1(S_accumulator[7:4], HEX1);
hex_7seg h0(S_accumulator[3:0], HEX0);
```

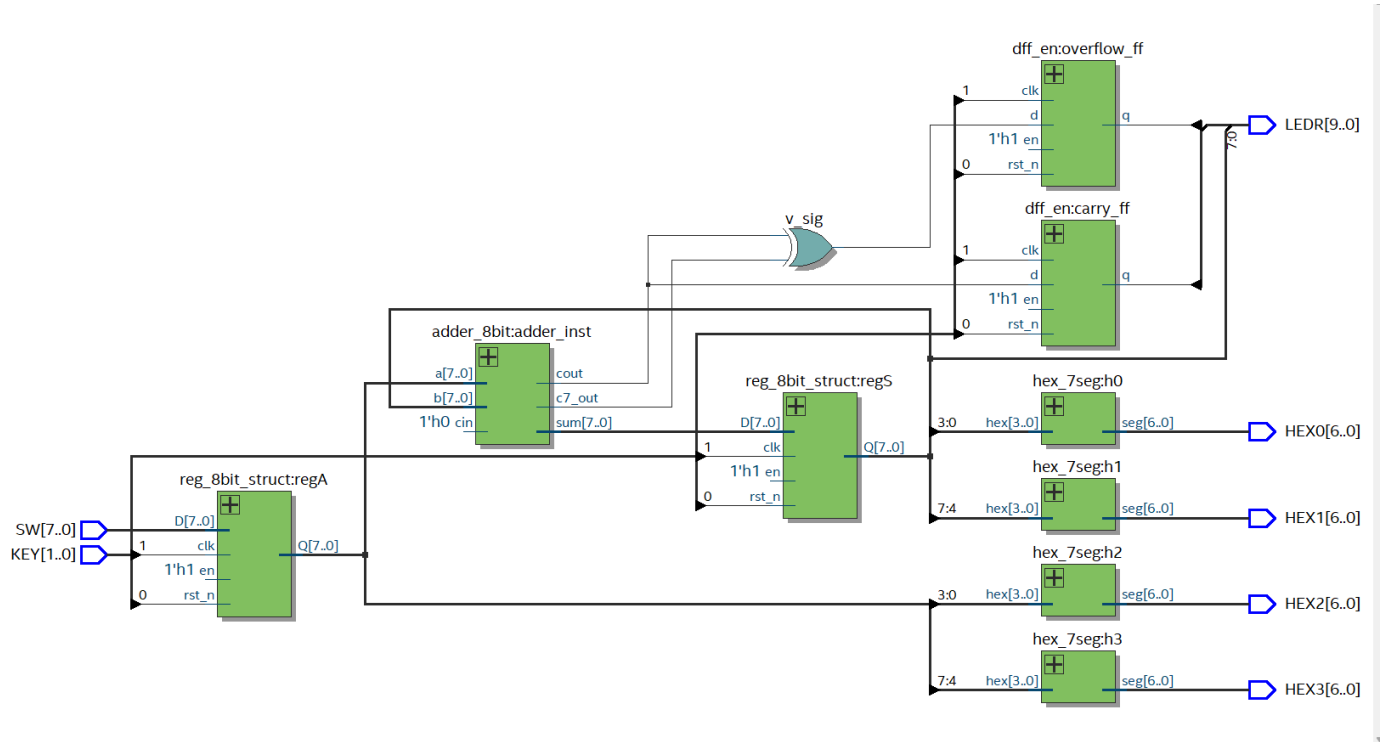
endmodule



Laboratory 1:

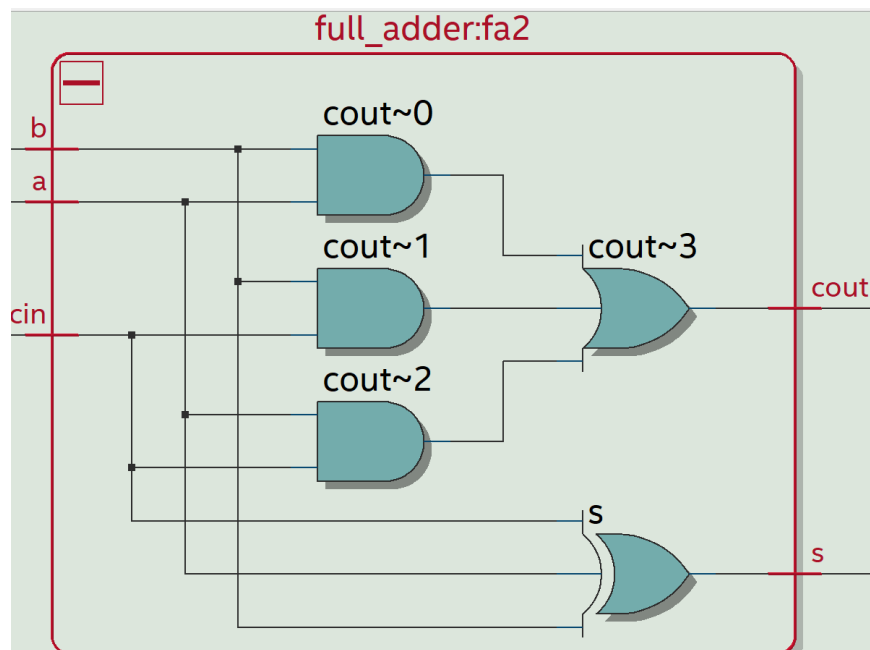
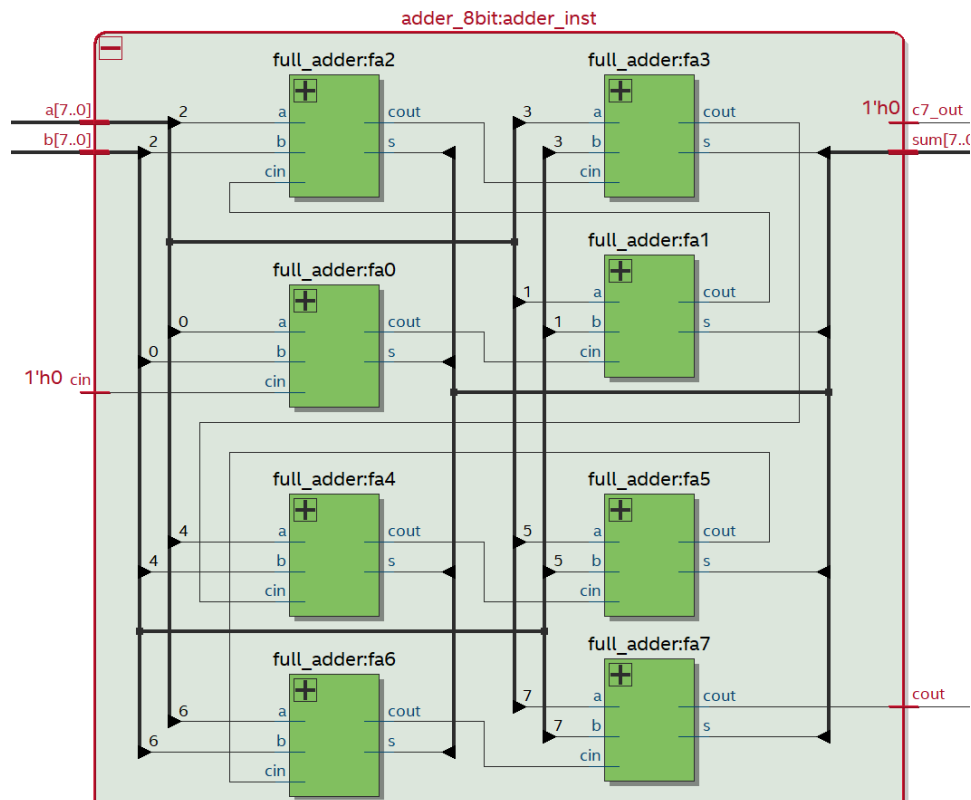
ADDERS, SUBTRACTORS AND MULTIPLIERS

RTL viewer



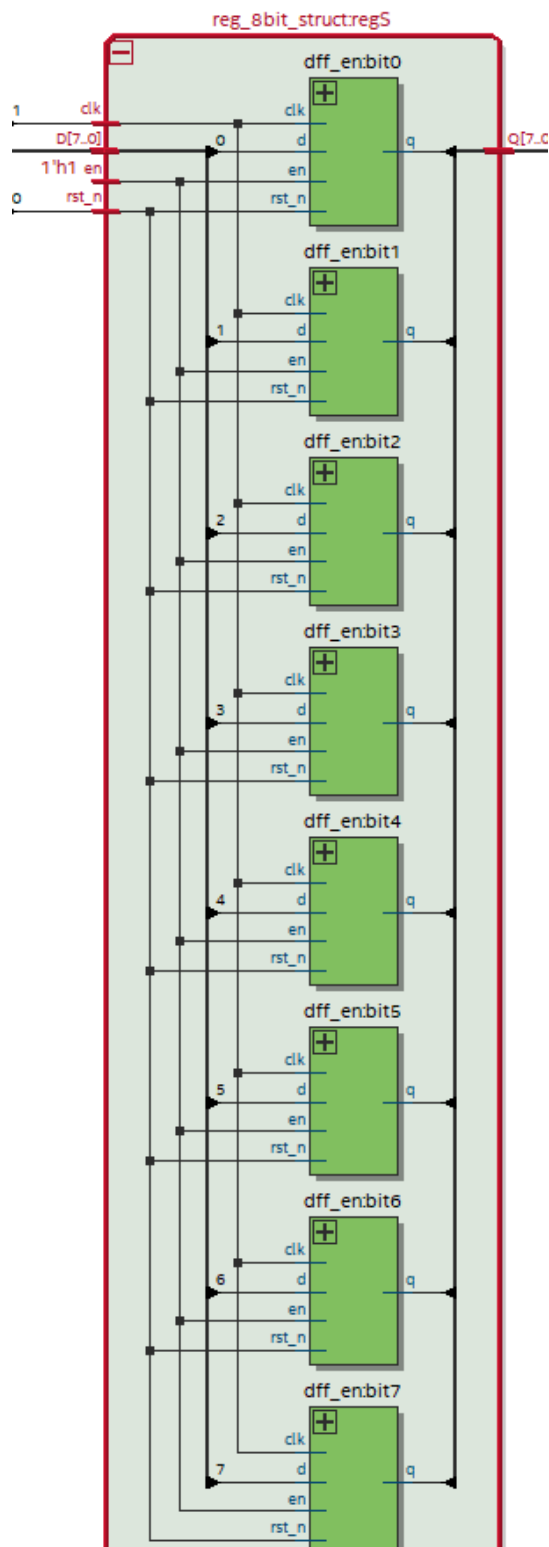
Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



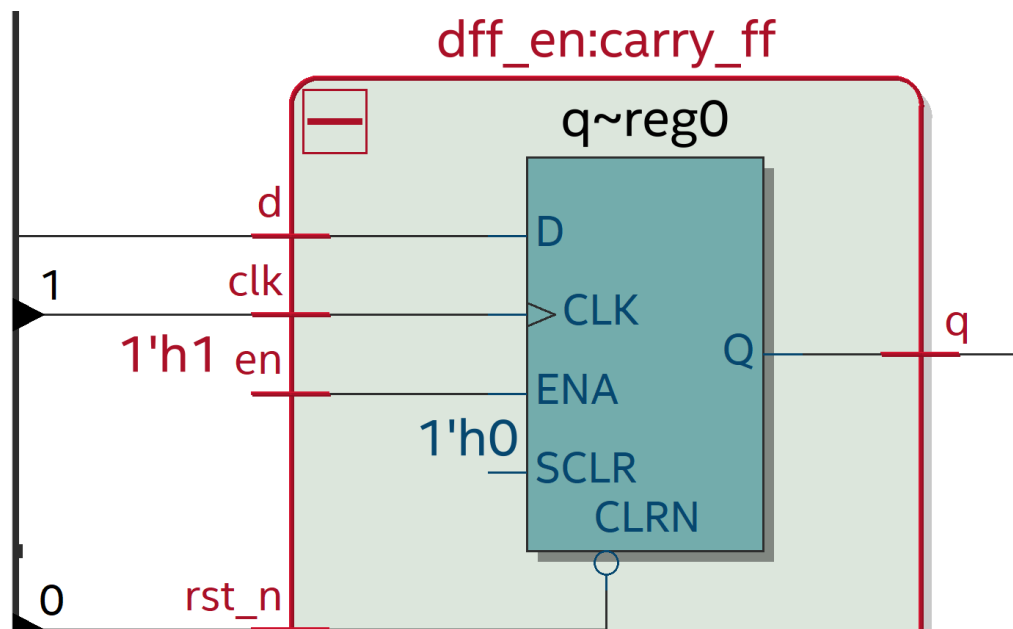
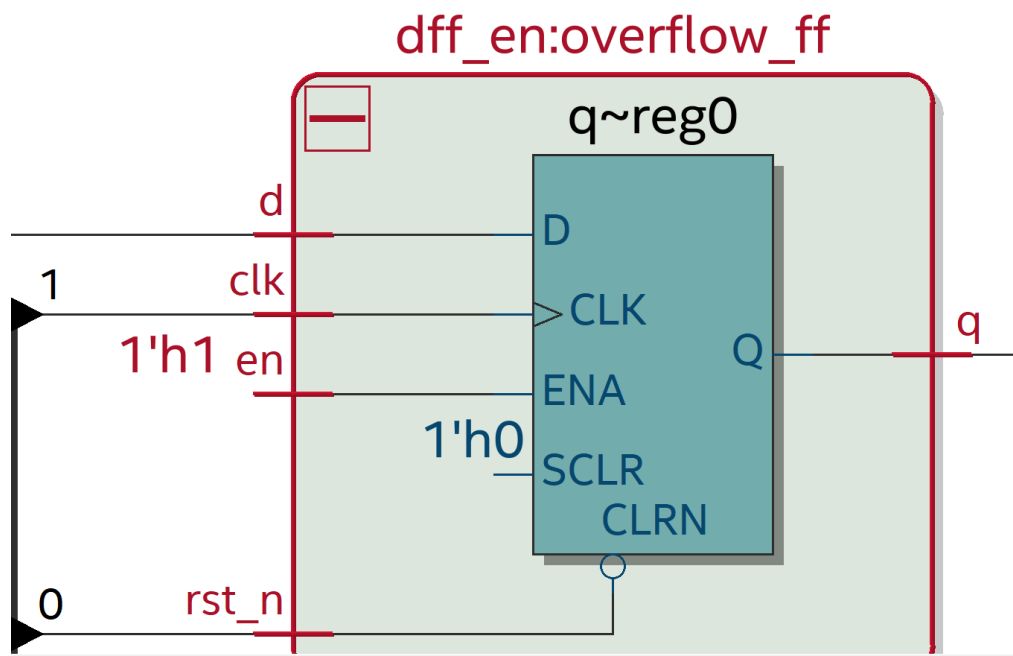
Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



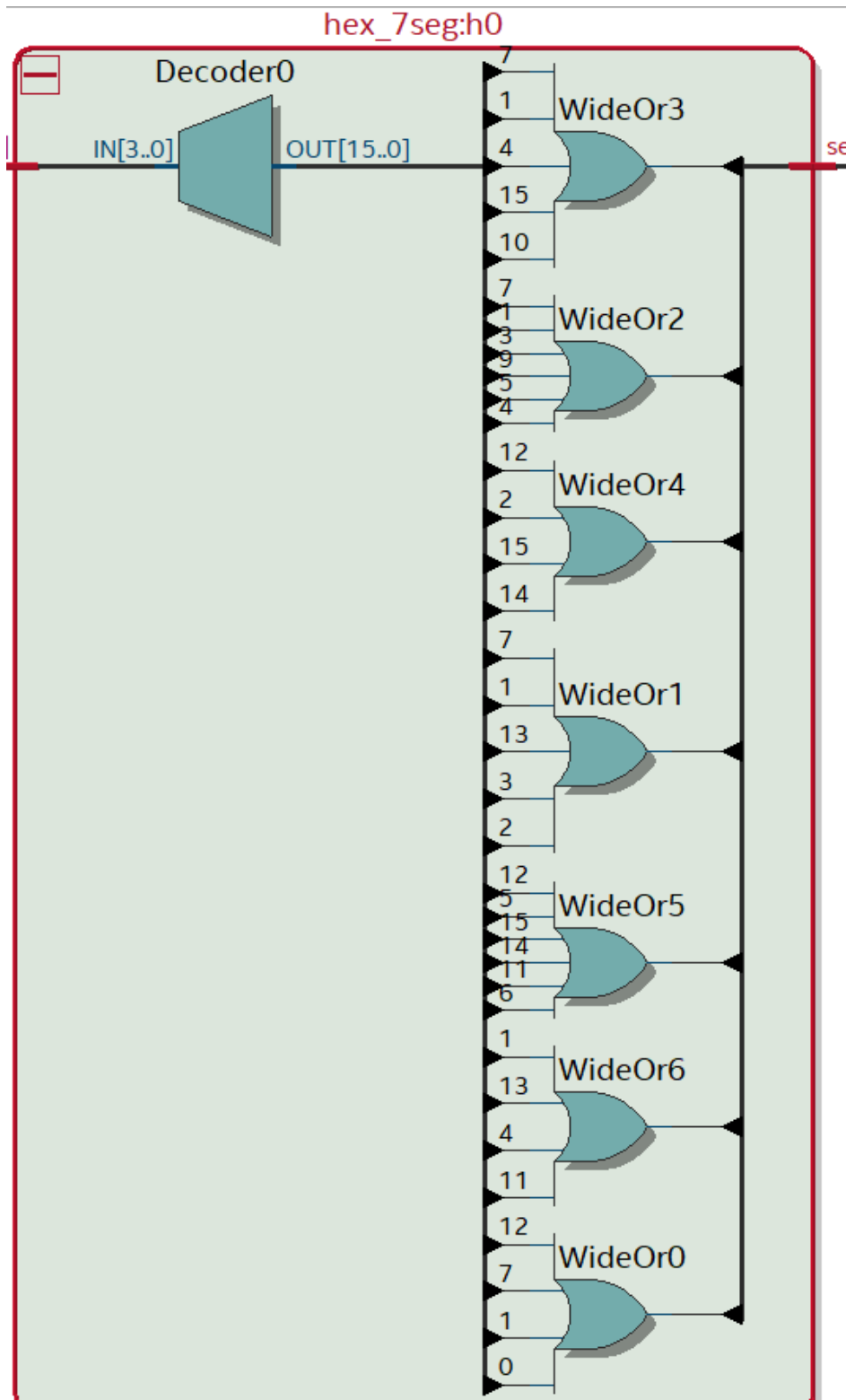
Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Testbench

```
`timescale 1ns/1ps

module lab1_tn1_tb();
    // --- 6 THÀNH PHẦN TRONG FIGURE 1 ---
    logic      clk;          // 1. Clock [cite: 22, 70]
    logic      rst;          // 2. Reset (tương ứng rst trong waveform) [cite: 67,
71]
    logic [7:0] A;           // 3. Input A (8-bit) [cite: 26, 72]
    logic [7:0] S;           // 4. Output Sum S (8-bit) [cite: 32, 81]
    logic      carry;        // 5. Carry flag [cite: 31, 82]
    logic      overflow;     // 6. Overflow flag [cite: 31, 83]

    // Kết nối với các cổng của kit FPGA (nếu cần)
    logic [9:0] LEDR;
    assign S      = LEDR[7:0]; // S hiển thị trên LEDR7-0 [cite: 91]
    assign carry  = LEDR[8];   // Carry hiển thị trên LEDR8 [cite: 91]
    assign overflow = LEDR[9]; // Overflow hiển thị trên LEDR9 [cite: 91]

    // Khởi tạo module Experiment 1 (UUT)
    lab1_tn1 uut (
        .SW(A),                // Gán SW7-0 cho đầu vào A [cite: 90]
        .KEY({clk, ~rst}),     // KEY1 là Clock, KEY0 là rst (active-low)
        [cite: 90]
        .LEDR(LEDR),
        .HEX0(), .HEX1(), .HEX2(), .HEX3()
    );

    // Tạo xung nhịp chu kỳ 10ns
    always #5 clk = ~clk;

    initial begin
        // --- KHỞI TẠO ---
        clk = 0;
        rst = 0;
        A   = 8'h00;

        // --- GIAI ĐOẠN RESET (Khớp với waveform của đề) ---
        #2  rst = 1; // Tín hiệu rst nhô cao
        #10 rst = 0; // Kết thúc reset, hệ thống bắt đầu tích lũy
        #5;

        // --- CẤP CÁC GIÁ TRỊ A THEO ĐỀ BÀI ---
        // Sử dụng negedge clk để thay đổi A trước khi cạnh dương đến
        @(negedge clk) A = 8'h70; // [cite: 73]
        @(negedge clk) A = 8'h25; //
        @(negedge clk) A = 8'h5A; //
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

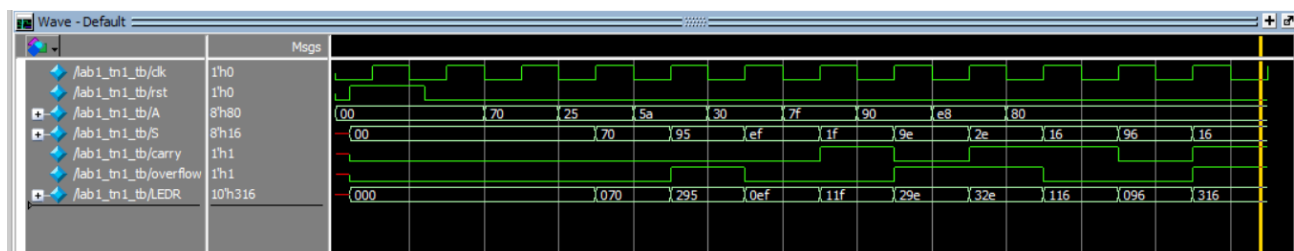
```
@(negedge clk) A = 8'h30; // [cite: 75]
@(negedge clk) A = 8'h7F; // [cite: 76]
@(negedge clk) A = 8'h90; // [cite: 77]
@(negedge clk) A = 8'hE8; // [cite: 78]
@(negedge clk) A = 8'h80; // [cite: 79]
@(negedge clk) A = 8'h80; // [cite: 80]

// Chờ để quan sát kết quả cuối cùng
repeat(3) @(posedge clk);

$display("Mô phỏng hoàn tất. Hãy kiểm tra của sổ Waveform!");
$stop;

end
endmodule
```

Simulation



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Kit



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

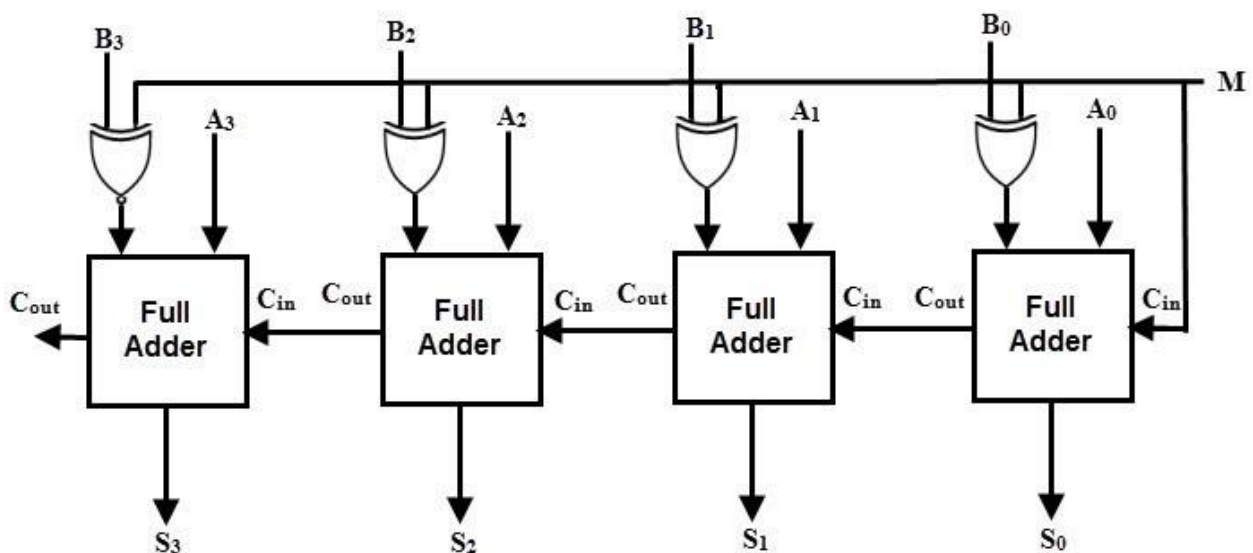
EXPERIMENT 2

Objective: Known how to program a system to add or subtract the value of an input A to itself repeatedly.

Requirement: Extend the circuit from Experiment 1 to be able to both add and subtract numbers. To do so, add an *add_sub* input to your circuit. When *add_sub* is 1, your circuit should subtract *A* from register *S*, and when *add_sub* is 0 your circuit should add *A* to register *S* as in experiment 1.

Instruction:

- There are two ways to implement that exercise:
 - If the 4-bit adder circuit is constructed from four instances of the FA, using the example 4-bit circuit:



- If the 4-bit adder entity is written by using a '+' sign in System Verilog, you need to modify the circuit using IF_ELSE statement.
- Connect input A to switches SW7–0, input *add_sub* to switch 9. Use KEY0 as an active-low asynchronous reset, and use KEY1 as a manual clock input. The sum from the adder should



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

be displayed on the red lights LEDR7–0, the registered carry signal should be displayed on LEDR8, and the registered overflow signal should be displayed on LEDR9.

Show the registered values of A and S as hexadecimal numbers on the 7-segment displays HEX3–2 and HEX1 – 0.

Check: Your report has to show two results:

- The waveform to prove the circuit works correctly. You must show all the cases of overflow flag and carry flag.
- The result of RTL viewer.



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Code

```
// --- Module giải mã 7-segment (Giữ nguyên) ---
module hex_7seg (
    input logic [3:0] hex,
    output logic [6:0] seg
);
    always_comb begin
        case (hex)
            4'h0: seg = 7'b1000000; 4'h1: seg = 7'b1111001;
            4'h2: seg = 7'b0100100; 4'h3: seg = 7'b0110000;
            4'h4: seg = 7'b0011001; 4'h5: seg = 7'b0010010;
            4'h6: seg = 7'b0000010; 4'h7: seg = 7'b1111000;
            4'h8: seg = 7'b0000000; 4'h9: seg = 7'b0010000;
            4'hA: seg = 7'b0001000; 4'hB: seg = 7'b0000011;
            4'hC: seg = 7'b1000110; 4'hD: seg = 7'b0100001;
            4'hE: seg = 7'b0000110; 4'hF: seg = 7'b0001110;
            default: seg = 7'b1111111;
        endcase
    end
endmodule

// --- Khối D-Flip Flop (Giữ nguyên) ---
module dff_en (
    input logic d, clk, rst_n, en,
    output logic q
);
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            q <= 1'b0;
        else if (en)
            q <= d;
    end
endmodule

// --- Thanh ghi 8-bit (Giữ nguyên) ---
module reg_8bit_struct (
    input logic [7:0] D,
    input logic clk, rst_n, en,
    output logic [7:0] Q
);
    dff_en bit0 (D[0], clk, rst_n, en, Q[0]);
    dff_en bit1 (D[1], clk, rst_n, en, Q[1]);
    dff_en bit2 (D[2], clk, rst_n, en, Q[2]);
    dff_en bit3 (D[3], clk, rst_n, en, Q[3]);
    dff_en bit4 (D[4], clk, rst_n, en, Q[4]);
    dff_en bit5 (D[5], clk, rst_n, en, Q[5]);
    dff_en bit6 (D[6], clk, rst_n, en, Q[6]);
endmodule
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
dff_en bit7 (D[7], clk, rst_n, en, Q[7]);
endmodule

// --- Full Adder (Giữ nguyên) ---
module full_adder (
    input logic a, b, cin,
    output logic s, cout
);
    assign s = a ^ b ^ cin;
    assign cout = (a & b) | (b & cin) | (a & cin);
endmodule

// --- Module Adder/Subtractor 8-bit ---
module adder_subtractor_8bit (
    input logic [7:0] a, b,
    input logic add_sub, // Tín hiệu điều khiển M [cite: 151]
    output logic [7:0] sum,
    output logic cout, // C8
    output logic c7_out // Carry vào bit đầu để tính Overflow
);
    logic [7:1] c;
    logic [7:0] b_xor;

    // Nếu add_sub = 1, đảo bit của b và cin = 1 để thực hiện bù 2
    assign b_xor = b ^ {8{add_sub}};

    full_adder fa0 (a[0], b_xor[0], add_sub, sum[0], c[1]);
    full_adder fa1 (a[1], b_xor[1], c[1], sum[1], c[2]);
    full_adder fa2 (a[2], b_xor[2], c[2], sum[2], c[3]);
    full_adder fa3 (a[3], b_xor[3], c[3], sum[3], c[4]);
    full_adder fa4 (a[4], b_xor[4], c[4], sum[4], c[5]);
    full_adder fa5 (a[5], b_xor[5], c[5], sum[5], c[6]);
    full_adder fa6 (a[6], b_xor[6], c[6], sum[6], c[7]);
    full_adder fa7 (a[7], b_xor[7], c[7], sum[7], cout);

    assign c7_out = c[7];
endmodule

// --- MODULE CHÍNH CHO EXPERIMENT 2 -
module lab1_tn2 (
    input logic [9:0] SW, // SW7-0: A, SW9: add_sub [cite: 175]
    input logic [1:0] KEY, // KEY0: rst_n, KEY1: clock
    output logic [9:0] LEDR, // LEDR7-0: S, LEDR8: Carry, LEDR9: Overflow
    [cite: 181]
    output logic [6:0] HEX0, HEX1, HEX2, HEX3 // HEX3-2: A, HEX1-0: S [cite:
182]
);
    // Tín hiệu nội bộ
    logic [7:0] A_reg, S_accumulator, next_res;
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
logic c7_sig, c8_sig, v_sig;
logic add_sub_reg; // TÍNH HIỆU MỚI ĐÃ THÊM [cite: 114]

// 1. Thanh ghi lưu giá trị A [cite: 110]
reg_8bit_struct regA (
    .D(SW[7:0]),
    .clk(KEY[1]),
    .rst_n(KEY[0]),
    .en(1'b1),
    .Q(A_reg)
);

// 2. THANH GHI CHO ADD_SUB (Đồng bộ hóa điều khiển)
dff_en add_sub_sync (
    .d(SW[9]),
    .clk(KEY[1]),
    .rst_n(KEY[0]),
    .en(1'b1),
    .q(add_sub_reg)
);

// 3. Bộ Cộng/Trừ sử dụng add_sub_reg [cite: 104, 106, 120]
adder_subtractor_8bit adder_sub_inst (
    .a(S_accumulator),
    .b(A_reg),
    .add_sub(add_sub_reg), // THAY ĐỔI TẠI ĐÂY
    .sum(next_res),
    .cout(c8_sig),
    .c7_out(c7_sig)
);

// 4. Phát hiện lỗi tràn ( $V = C7 \wedge C8$ ) [cite: 45, 146]
assign v_sig = c7_sig ^ c8_sig;

// 5. Thanh ghi tích lũy S [cite: 116]
reg_8bit_struct regS (
    .D(next_res),
    .clk(KEY[1]),
    .rst_n(KEY[0]),
    .en(1'b1),
    .Q(S_accumulator)
);

// 6. Chốt Carry và Overflow vào LEDR8 và LEDR9 [cite: 181]
dff_en carry_ff      (c8_sig, KEY[1], KEY[0], 1'b1, LEDR[8]);
dff_en overflow_ff   (v_sig,   KEY[1], KEY[0], 1'b1, LEDR[9]);

assign LEDR[7:0] = S_accumulator;
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
// 7. Hiện thị lên 7-segment [cite: 182]
hex_7seg h3(A_reg[7:4], HEX3);
hex_7seg h2(A_reg[3:0], HEX2);
hex_7seg h1(S_accumulator[7:4], HEX1);
hex_7seg h0(S_accumulator[3:0], HEX0);

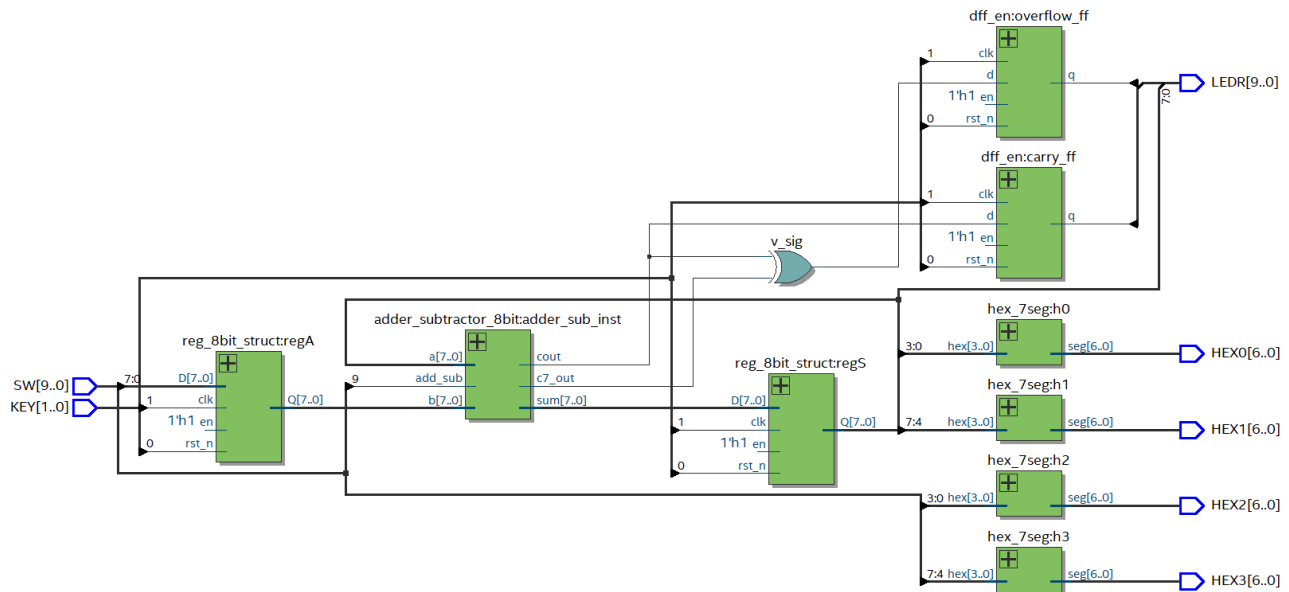
endmodule
```



Laboratory 1:

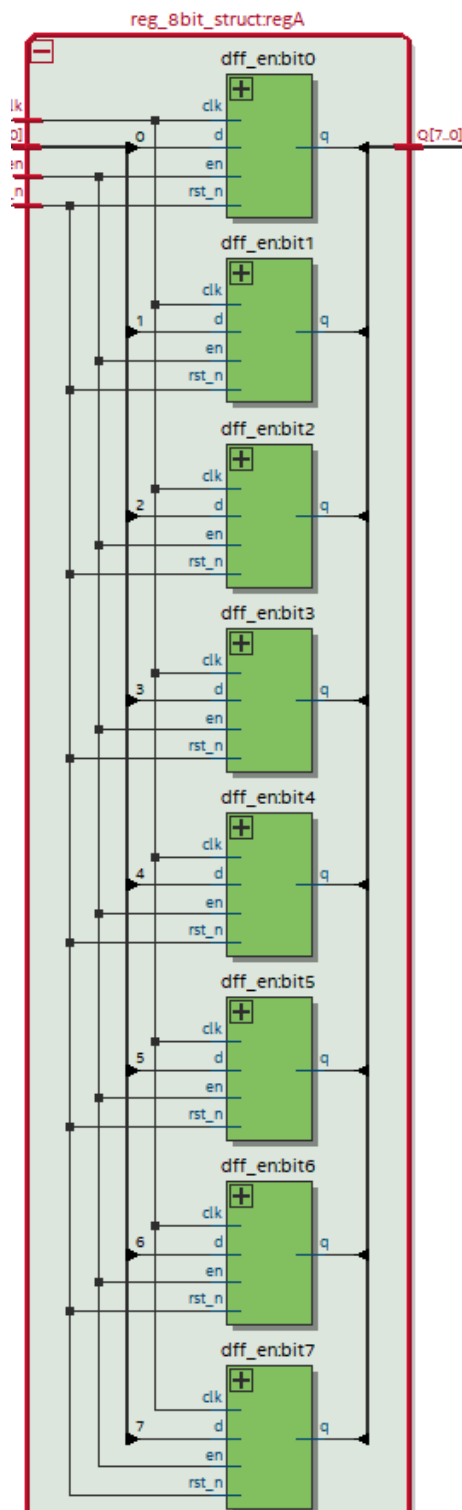
ADDERS, SUBTRACTORS AND MULTIPLIERS

RTL viewer



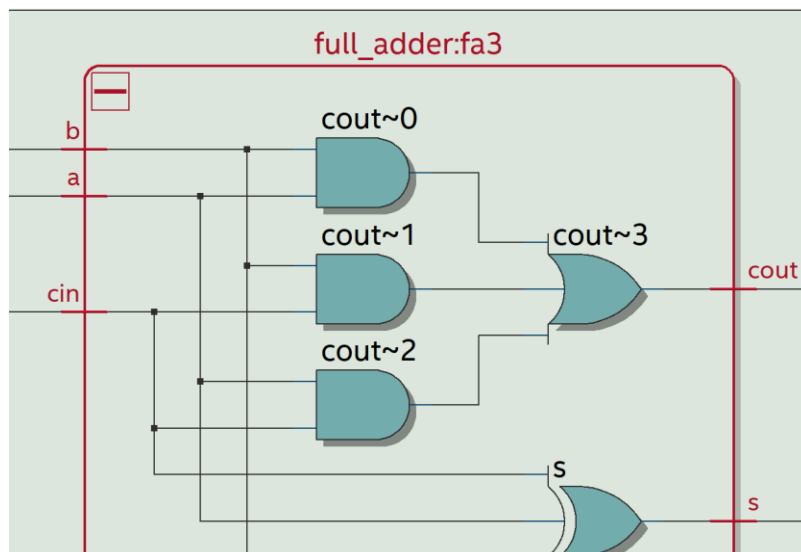
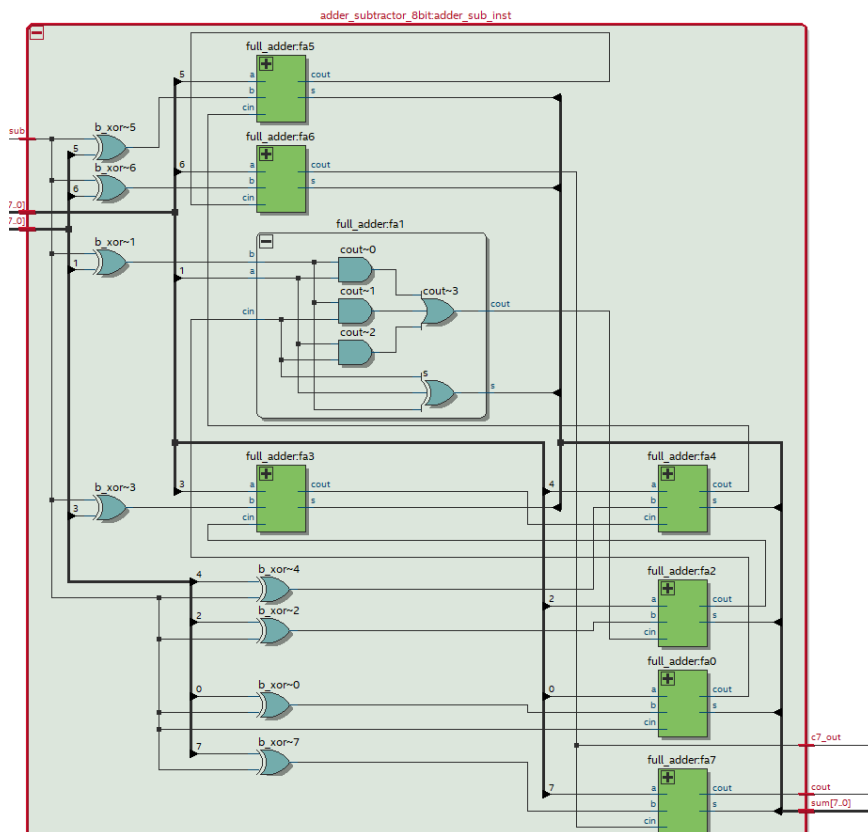
Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



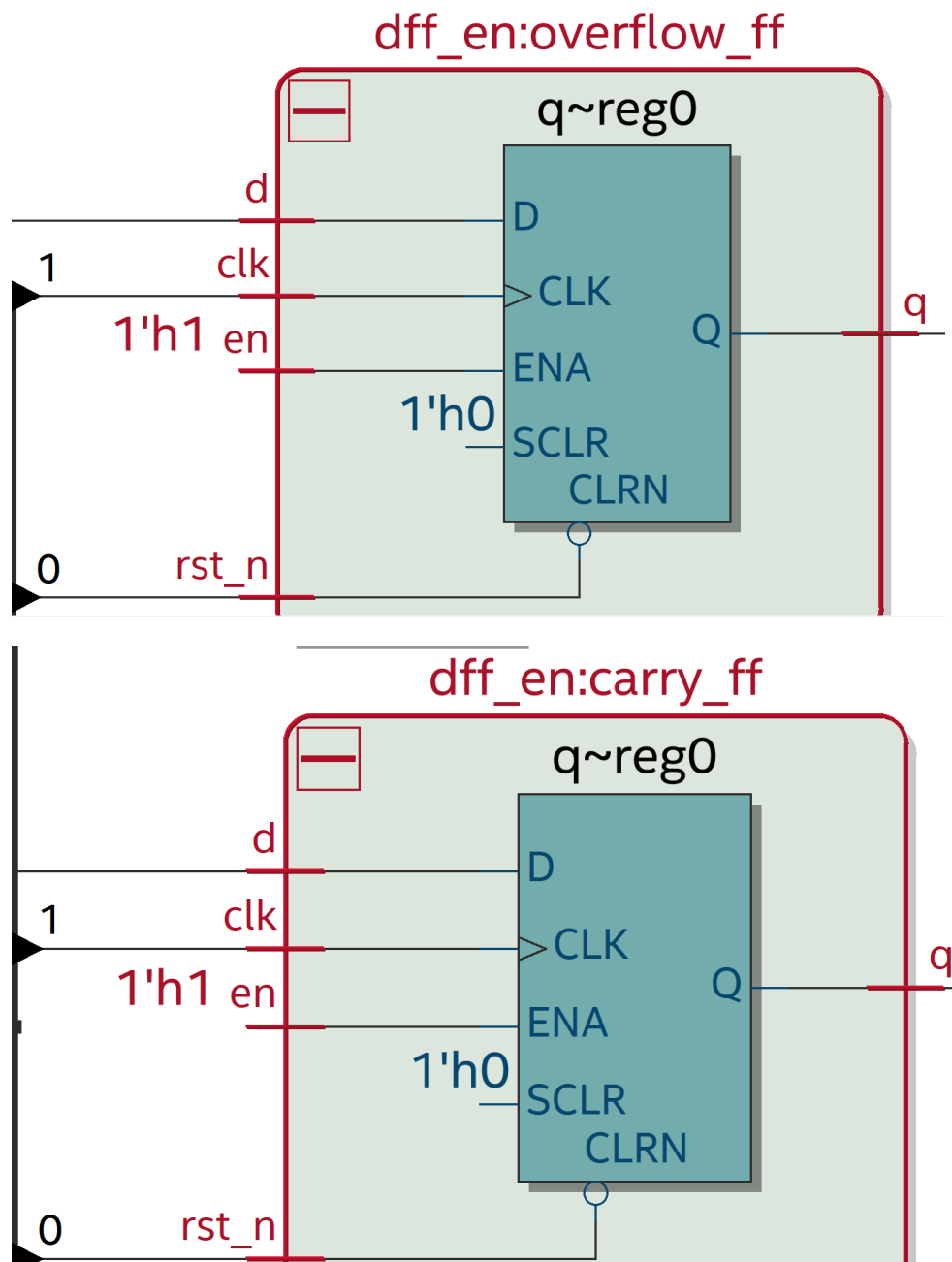
Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



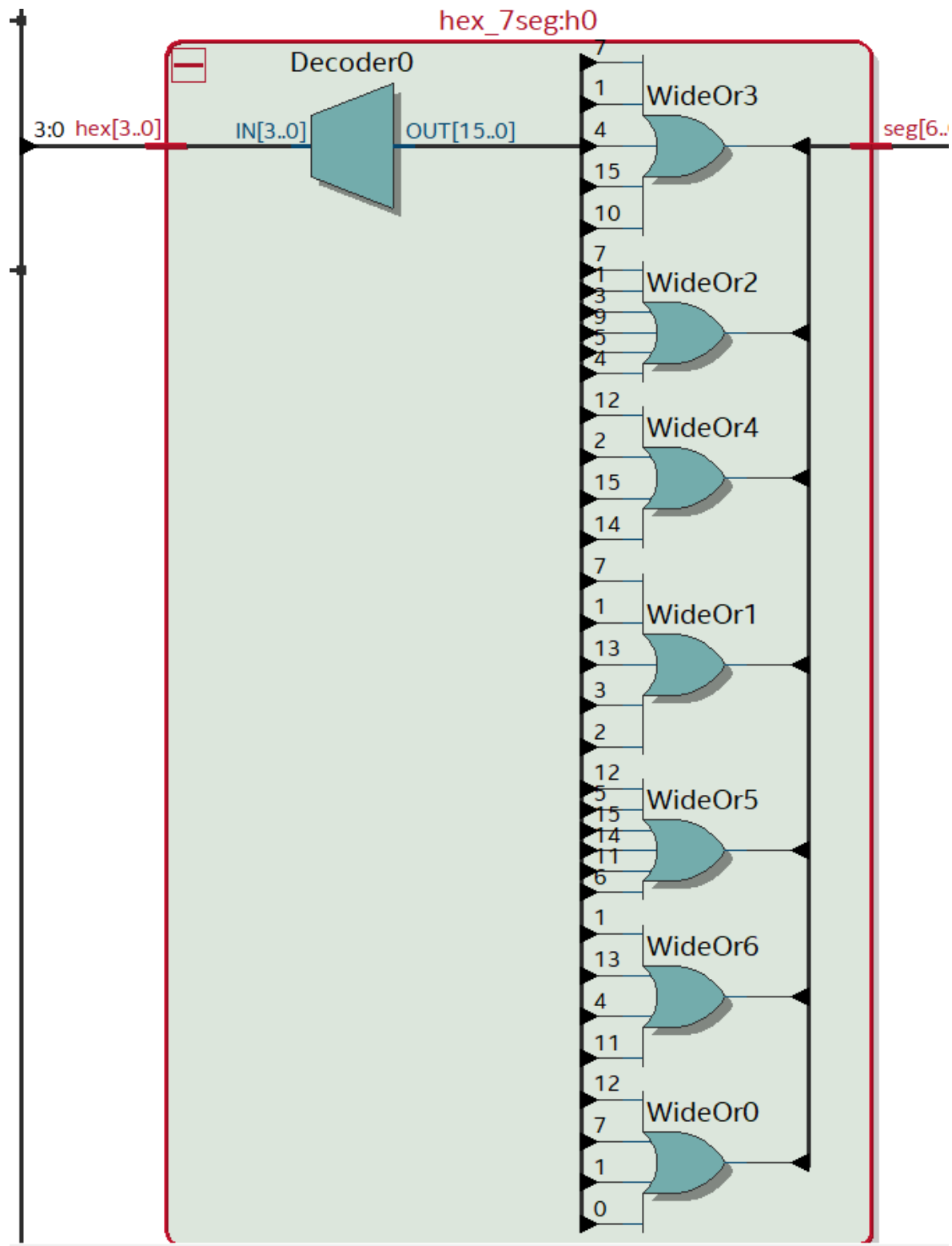
Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Testbench

```
`timescale 1ns/1ps

module lab1_tn2_tb();
    // --- CÁC THÀNH PHẦN HIỂN THỊ TRONG FIGURE & DIAGRAM ---
    logic      Clock;      // KEY1
    logic      rst;        // KEY0 (Active-low reset)
    logic      add_sub;    // SW9
    logic [7:0] A;          // SW7-0
    logic [7:0] S;          // LEDR7-0
    logic      carry;      // LEDR8
    logic      overflow;   // LEDR9

    // Kết nối tín hiệu cho Module chính
    logic [9:0] SW_in;
    logic [1:0] KEY_in;
    logic [9:0] LEDR_out;

    assign SW_in[7:0] = A;
    assign SW_in[9]   = add_sub;
    assign SW_in[8]    = 1'b0;
    assign KEY_in[1]   = Clock;
    assign KEY_in[0]   = rst;

    assign S           = LEDR_out[7:0];
    assign carry        = LEDR_out[8];
    assign overflow     = LEDR_out[9];

    // Khởi tạo Unit Under Test (UUT)
    lab1_tn2 uut (
        .SW(SW_in), .KEY(KEY_in), .LEDR(LEDR_out),
        .HEX0(), .HEX1(), .HEX2(), .HEX3()
    );

    // Tạo xung nhịp chu kỳ 10ns
    always #5 Clock = ~Clock;

    initial begin
        // 1. Khởi tạo ban đầu
        Clock    = 0;
        rst      = 1;
        add_sub  = 0;
        A        = 8'h00;

        // 2. Thực hiện Reset (Khớp với đoạn đầu rst nhô cao trong diagram)
        #2 rst = 0;
        #10 rst = 1;
    end
endmodule
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

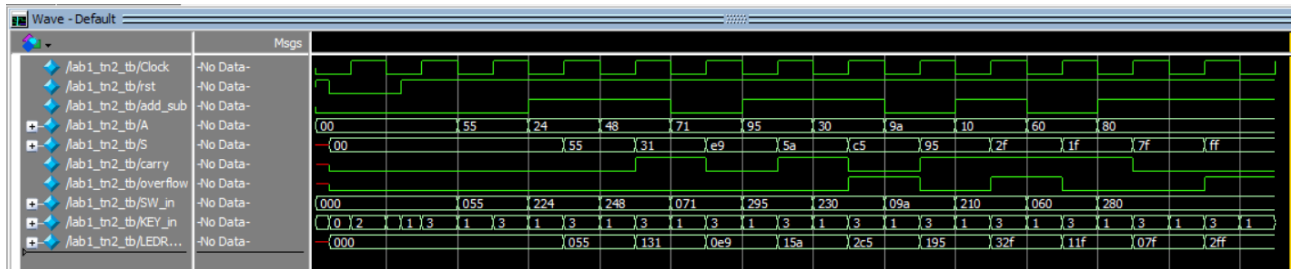
```
#5;

// 3. Chuỗi giá trị A và add_sub
@(negedge Clock) begin A = 8'h55; add_sub = 0; end // Bước 1: Cộng 0x55
@(negedge Clock) begin A = 8'h24; add_sub = 1; end // Bước 2: Cộng 0x24
@(negedge Clock) begin A = 8'h48; add_sub = 1; end // Bước 3: Trừ 0x48
@(negedge Clock) begin A = 8'h71; add_sub = 0; end // Bước 4: Cộng 0x71
@(negedge Clock) begin A = 8'h95; add_sub = 1; end // Bước 5: Trừ 0x95
@(negedge Clock) begin A = 8'h30; add_sub = 1; end // Bước 6: Cộng 0x30
@(negedge Clock) begin A = 8'h9A; add_sub = 0; end // Bước 7: Trừ 0x9A
@(negedge Clock) begin A = 8'h10; add_sub = 1; end // Bước 8: Cộng 0x10
@(negedge Clock) begin A = 8'h60; add_sub = 0; end // Bước 9: Trừ 0x60
@(negedge Clock) begin A = 8'h80; add_sub = 1; end // Bước 10: Cộng 0x80

repeat(3) @(posedge Clock);
$display("Simulation Experiment 2 hoàn tất!");
$stop;

end
endmodule
```

Simulation



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Kit Subtract

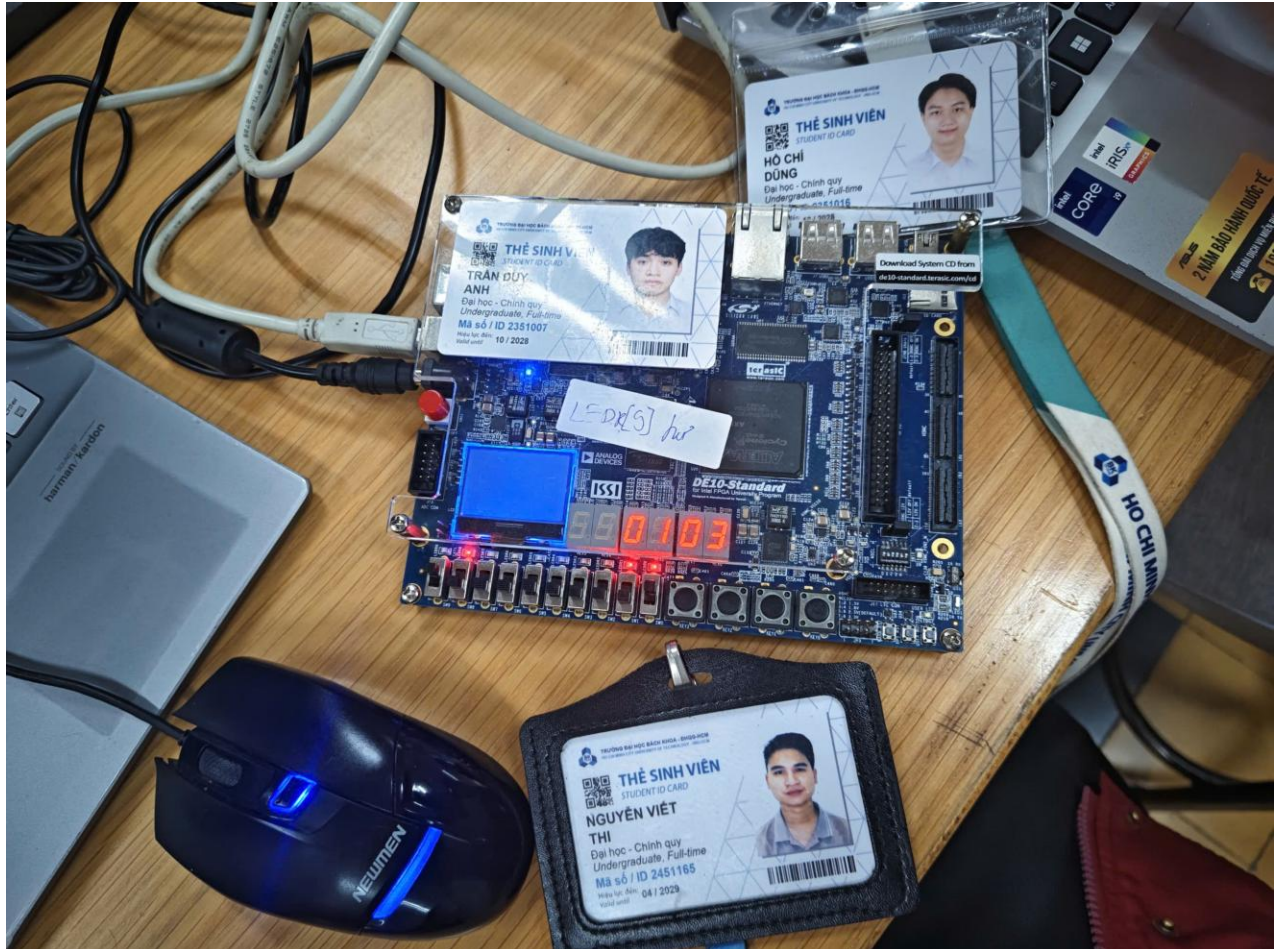


Figure: $C_{in} = 1$, Input = 1, Result = 3

Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Figure: $C_{in} = 1$, Input = 1, Result = 2



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Add

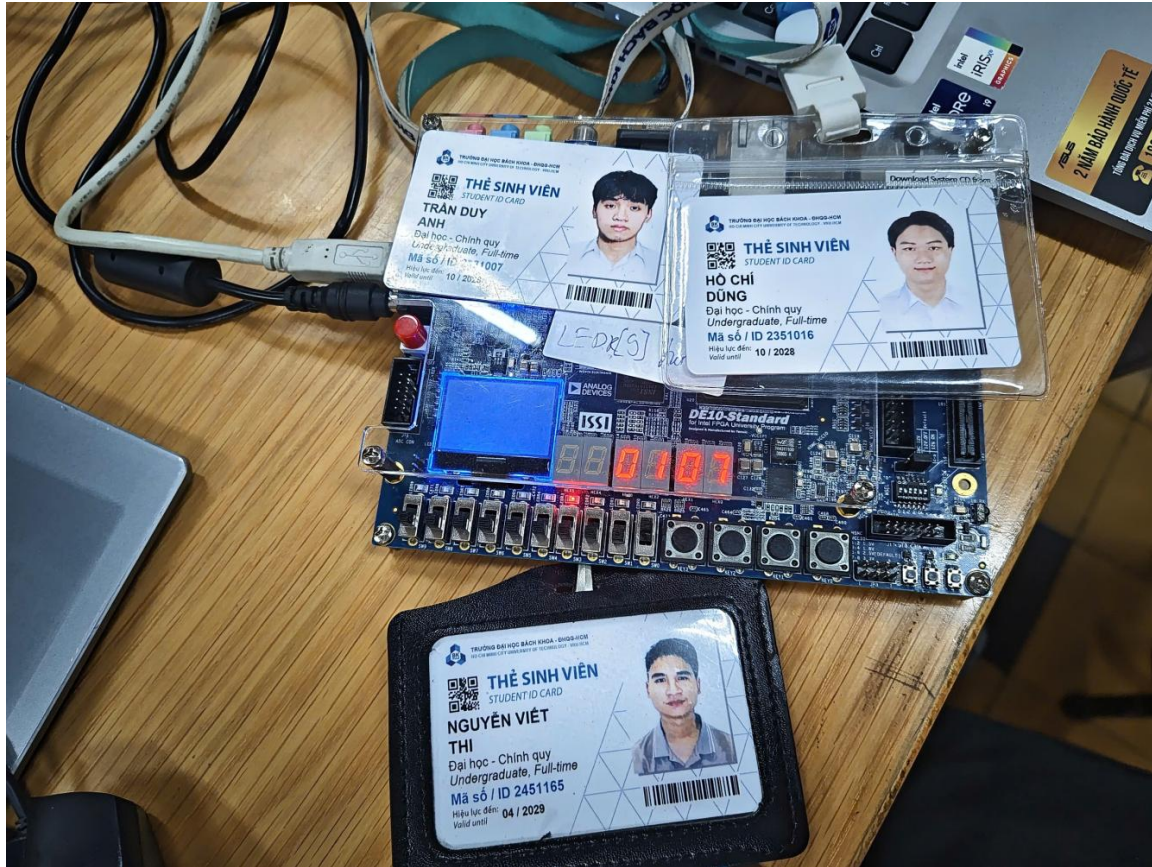


Figure: $C_{in} = 0$, Input = 1, Result = 7



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Figure: $C_{in} = 0$, Input = 1, Result = 8



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Figure: $C_{in} = 0$, Input = 1, Result = 9



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

EXPERIMENT 3

Objective: Known how to program 8x8 multiplier circuit with registered inputs and outputs.

Requirement: The circuit in Figure 2 shows an 8 x 8 multiplier circuit with registered inputs and outputs. Write System Verilog code to implement this system.

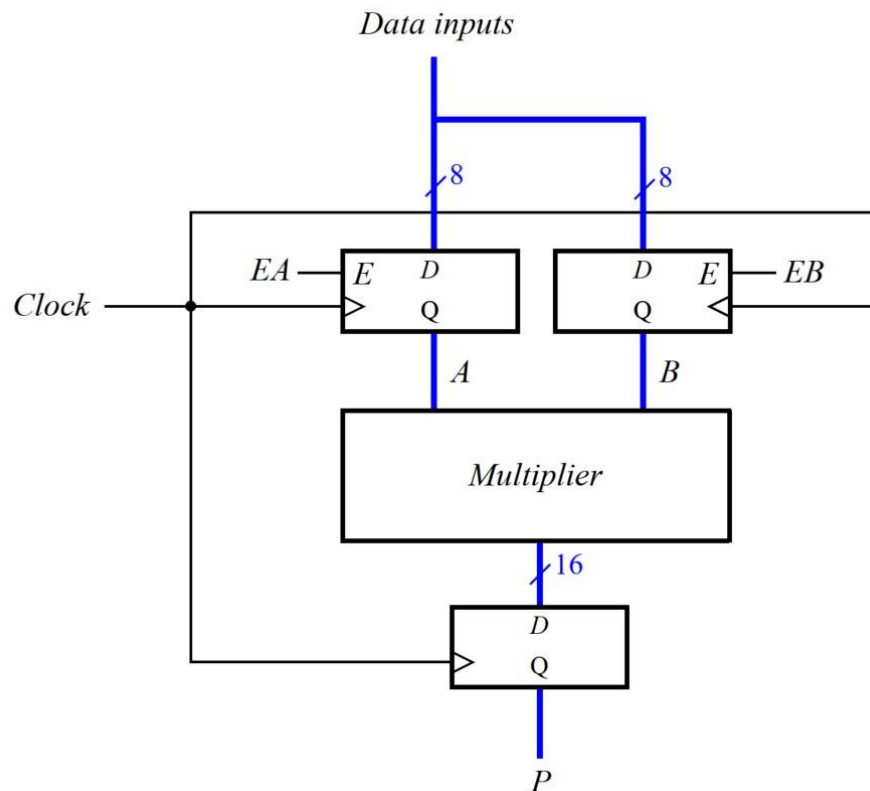


Figure 2: A registered multiplier circuit.

Instruction:

- In exercise 2, pre lab 3, you have write the multiplier. Write System Verilog module that instantiates the instances of this multiplier and three registers to describe the circuit given in Figure 2.
- Use switches SW7–0 to provide the data inputs to the circuit. Use SW9 as the enable signal EA for register A, and use SW8 as the enable for register B. When SW9 = 1 display the



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

contents of register A on the red lights LEDR, and display the contents of register B on these lights when SW8 = 1. Use KEY0 as a synchronous reset input, and use KEY1 as a manual clock signal. Show the product $P = A \times B$ as a hexadecimal number on the 7-segment displays HEX3-0.

- Using waveform, test the functionality of your design by inputting various data values and observing the generated products.

Check: Your report has to show two results:

- The waveform to prove the circuit works correctly.
- The result of RTL viewer.



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Code

```
module ex3(
    input logic [1:0] KEY,
    input logic [9:0] SW,
    output logic [9:0] LEDR,
    output logic [6:0] HEX0,
    output logic [6:0] HEX1,
    output logic [6:0] HEX2,
    output logic [6:0] HEX3
);

assign LEDR[7:0] = SW[7:0];

logic [15:0] reg_P;

top_mult multiply(
    .clk (KEY[1]),
    .rst (KEY[0]),
    .En_A (SW[9]),
    .En_B (SW[8]),
    .A (SW[7:0]),
    .B (SW[7:0]),
    .reg_P (reg_P)
);

led7seg_decoder hexP0(
    .i_data (reg_P[3:0]),
    .o_hex (HEX0)
);

led7seg_decoder hexP1(
    .i_data (reg_P[7:4]),
    .o_hex (HEX1)
);

led7seg_decoder hexP2(
    .i_data (reg_P[11:8]),
    .o_hex (HEX2)
);

led7seg_decoder hexP3(
    .i_data (reg_P[15:12]),
    .o_hex (HEX3)
);

endmodule
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
module top_mult(
    input logic clk,
    input logic rst,
    input logic En_A,
    input logic En_B,
    input logic [7:0] A,
    input logic [7:0] B,
    output logic [15:0] reg_P
);

logic [7:0] reg_A;
logic [7:0] reg_B;
logic [15:0] p;

eight_bit_register Register_A_mult(
    .clk (clk),
    .rst (rst),
    .D (En_A ? A : reg_A),
    .Q (reg_A)
);

eight_bit_register Register_B_mult(
    .clk (clk),
    .rst (rst),
    .D (En_B ? B : reg_B),
    .Q (reg_B)
);

eight_bit_multiplier Multiplier(
    .A (reg_A),
    .B (reg_B),
    .P (p)
);

register_16_bit Register_P(
    .clk (clk),
    .rst (rst),
    .D (p),
    .Q (reg_P)
);

endmodule

module eight_bit_multiplier(
    input logic [7:0] A,
    input logic [7:0] B,
    output logic [15:0] P
);

logic [7:0] sub_Sum0;
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
logic [7:0] sub_Sum1;
logic [7:0] sub_Sum2;
logic [7:0] sub_Sum3;
logic [7:0] sub_Sum4;
logic [7:0] sub_Sum5;
logic [7:0] sub_Sum6;

logic Cout0;
logic Cout1;
logic Cout2;
logic Cout3;
logic Cout4;
logic Cout5;
logic Cout6;

logic V0;
logic V1;
logic V2;
logic V3;
logic V4;
logic V5;
logic V6;

eight_bit_adder sub_add0(
    .A (A[7:1] & {8{B[0]}}),
    .B (A[7:0] & {8{B[1]}}),
    .C_in (0),
    .S (sub_Sum0),
    .C_out (Cout0),
    .V (V0)
);
assign P[0] = A[0] & B[0];
assign P[1] = sub_Sum0[0];

eight_bit_adder sub_add1(
    .A ({Cout0, sub_Sum0[7:1]}),
    .B (A[7:0] & {8{B[2]}}),
    .C_in (0),
    .S (sub_Sum1),
    .C_out (Cout1),
    .V (V1)
);
assign P[2] = sub_Sum1[0];

eight_bit_adder sub_add2(
    .A ({Cout1, sub_Sum1[7:1]}),
    .B (A[7:0] & {8{B[3]}}),
    .C_in (0),
    .S (sub_Sum2),
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
.C_out (Cout2),
.V (V2)
);
assign P[3] = sub_Sum2[0];

eight_bit_adder sub_add3(
.A ({Cout2, sub_Sum2[7:1]}),
.B (A[7:0] & {8{B[4]}}),
.C_in (0),
.S (sub_Sum3),
.C_out (Cout3),
.V (V3)
);
assign P[4] = sub_Sum3[0];

eight_bit_adder sub_add4(
.A ({Cout3, sub_Sum3[7:1]}),
.B (A[7:0] & {8{B[5]}}),
.C_in (0),
.S (sub_Sum4),
.C_out (Cout4),
.V (V4)
);
assign P[5] = sub_Sum4[0];

eight_bit_adder sub_add5(
.A ({Cout4, sub_Sum4[7:1]}),
.B (A[7:0] & {8{B[6]}}),
.C_in (0),
.S (sub_Sum5),
.C_out (Cout5),
.V (V5)
);
assign P[6] = sub_Sum5[0];

eight_bit_adder sub_add6(
.A ({Cout5, sub_Sum5[7:1]}),
.B (A[7:0] & {8{B[7]}}),
.C_in (0),
.S (sub_Sum6),
.C_out (Cout6),
.V (V6)
);
assign P[15:7] = {Cout6, sub_Sum6};

endmodule
module register_16_bit(
    input logic clk,
    input logic rst,
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
    input logic [15:0] D,
    output logic [15:0] Q
);
always_ff @( posedge clk or negedge rst ) begin
    if(!rst)
        Q <= 16'b0;
    else
        Q <= D;
end
endmodule
module eight_bit_adder(
    input logic [7:0] A,
    input logic [7:0] B,
    input logic C_in,
    output logic [7:0] S,
    output logic C_out,
    output logic V
);

logic [8:0] carry;

full_adder add0(
    .a (A[0]),
    .b (B[0]^C_in),
    .Cin (carry[0]),
    .s (S[0]),
    .Cout (carry[1])
);

full_adder add1(
    .a (A[1]),
    .b (B[1]^C_in),
    .Cin (carry[1]),
    .s (S[1]),
    .Cout (carry[2])
);

full_adder add2(
    .a (A[2]),
    .b (B[2]^C_in),
    .Cin (carry[2]),
    .s (S[2]),
    .Cout (carry[3])
);

full_adder add3(
    .a (A[3]),
    .b (B[3]^C_in),
    .Cin (carry[3]),
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
.s (S[3]),
.Cout (carry[4])
);

full_adder add4(
.a (A[4]),
.b (B[4]^C_in),
.Cin (carry[4]),
.s (S[4]),
.Cout (carry[5])
);

full_adder add5(
.a (A[5]),
.b (B[5]^C_in),
.Cin (carry[5]),
.s (S[5]),
.Cout (carry[6])
);

full_adder add6(
.a (A[6]),
.b (B[6]^C_in),
.Cin (carry[6]),
.s (S[6]),
.Cout (carry[7])
);

full_adder add7(
.a (A[7]),
.b (B[7]^C_in),
.Cin (carry[7]),
.s (S[7]),
.Cout (carry[8])
);

assign carry[0] = C_in;
assign C_out = carry[8];
assign V = carry[8] ^ carry[7];

endmodule
module led7seg_decoder(
input logic [3:0] i_data,
output logic [6:0] o_hex
);

always_comb begin
unique case(i_data)
4'd0: o_hex = 7'b1000000;
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
4'd1: o_hex = 7'b1111001;
4'd2: o_hex = 7'b0100100;
4'd3: o_hex = 7'b0110000;
4'd4: o_hex = 7'b0011001;
4'd5: o_hex = 7'b0010010;
4'd6: o_hex = 7'b0000010;
4'd7: o_hex = 7'b1111000;
4'd8: o_hex = 7'b0000000;
4'd9: o_hex = 7'b0010000;
4'd10: o_hex = 7'b0001000;
4'd11: o_hex = 7'b0000011;
4'd12: o_hex = 7'b1000110;
4'd13: o_hex = 7'b0100001;
4'd14: o_hex = 7'b0000110;
4'd15: o_hex = 7'b0001110;
default: o_hex = 7'b0111111;
endcase
end
endmodule
module full_adder(
    input a, b, Cin,
    output s, Cout
);
assign s = a^b^Cin;
assign Cout = a&b | (a^b)&Cin;
endmodule
module eight_bit_register(
    input logic clk,
    input logic rst,
    input logic [7:0] D,
    output logic [7:0] Q
);

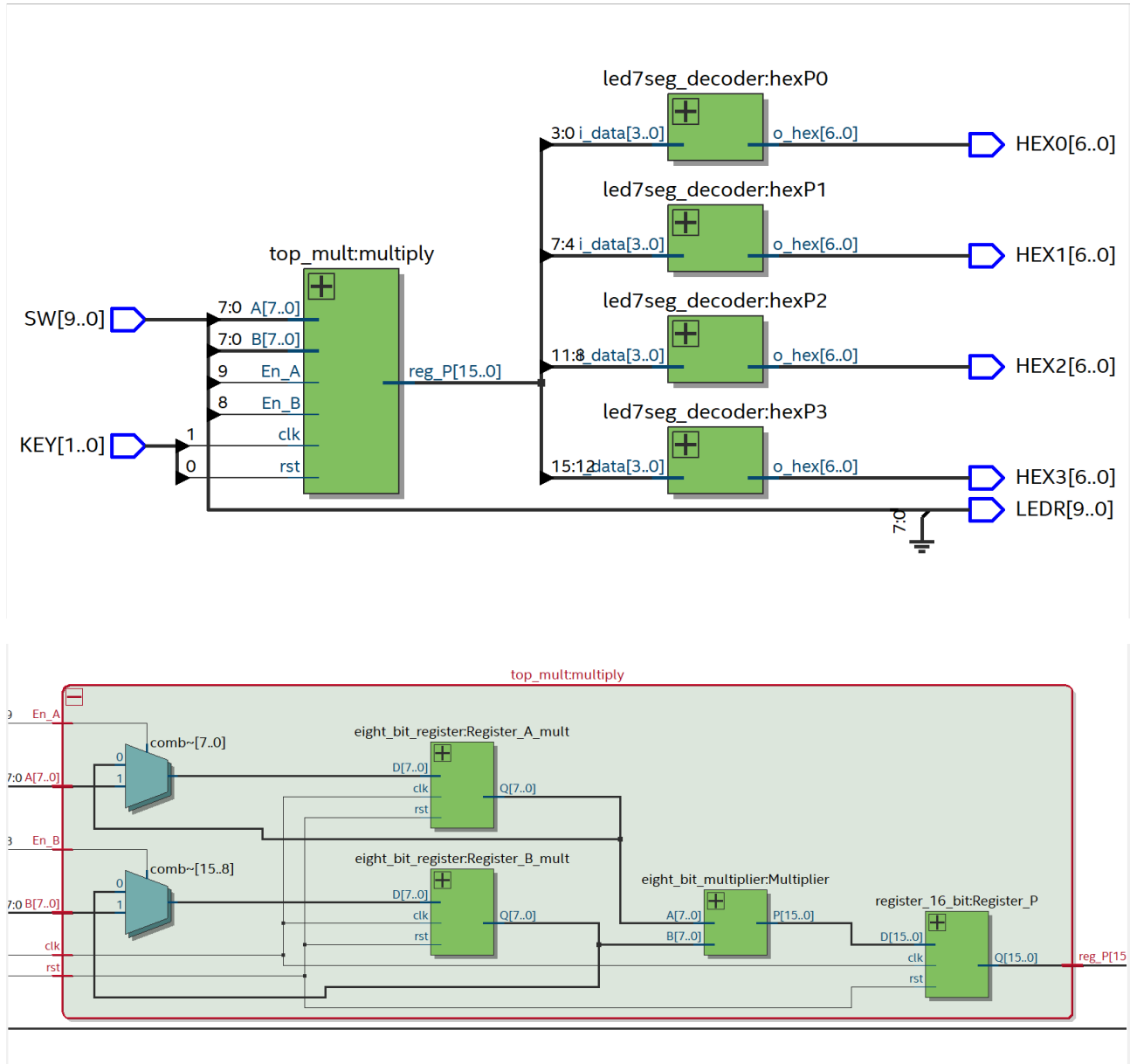
always_ff @( posedge clk or negedge rst ) begin
    if(!rst)
        Q <= 8'b0;
    else
        Q <= D;
    end
end
endmodule
```



Laboratory 1:

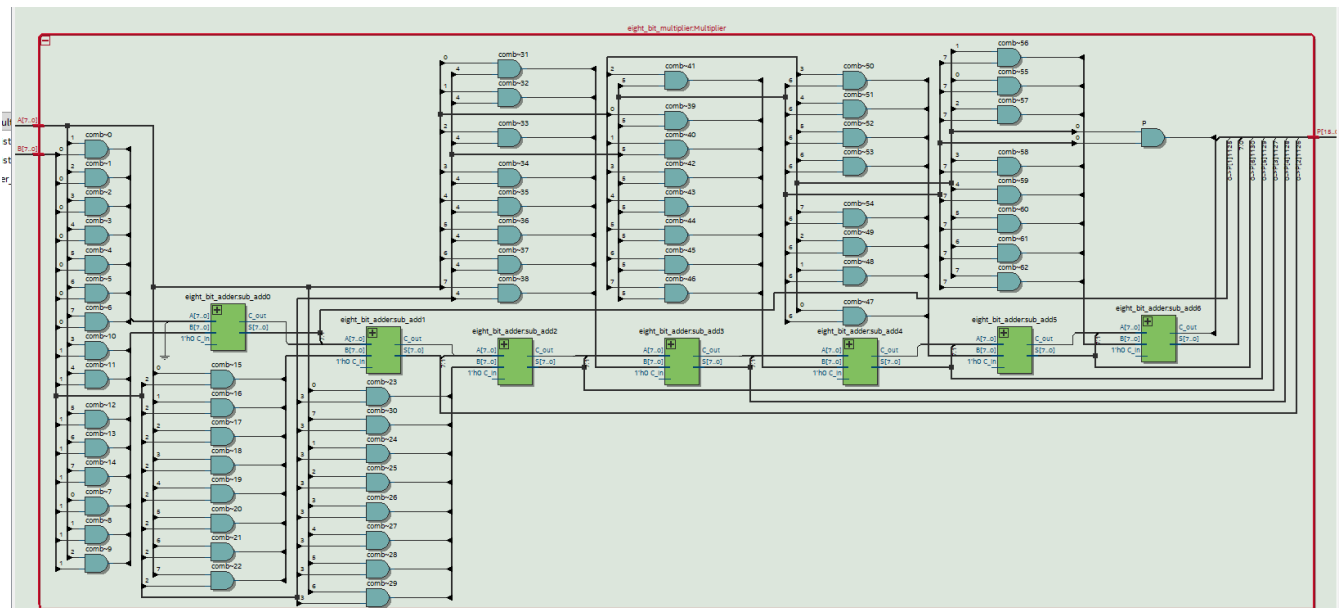
ADDERS, SUBTRACTORS AND MULTIPLIERS

RTL viewer



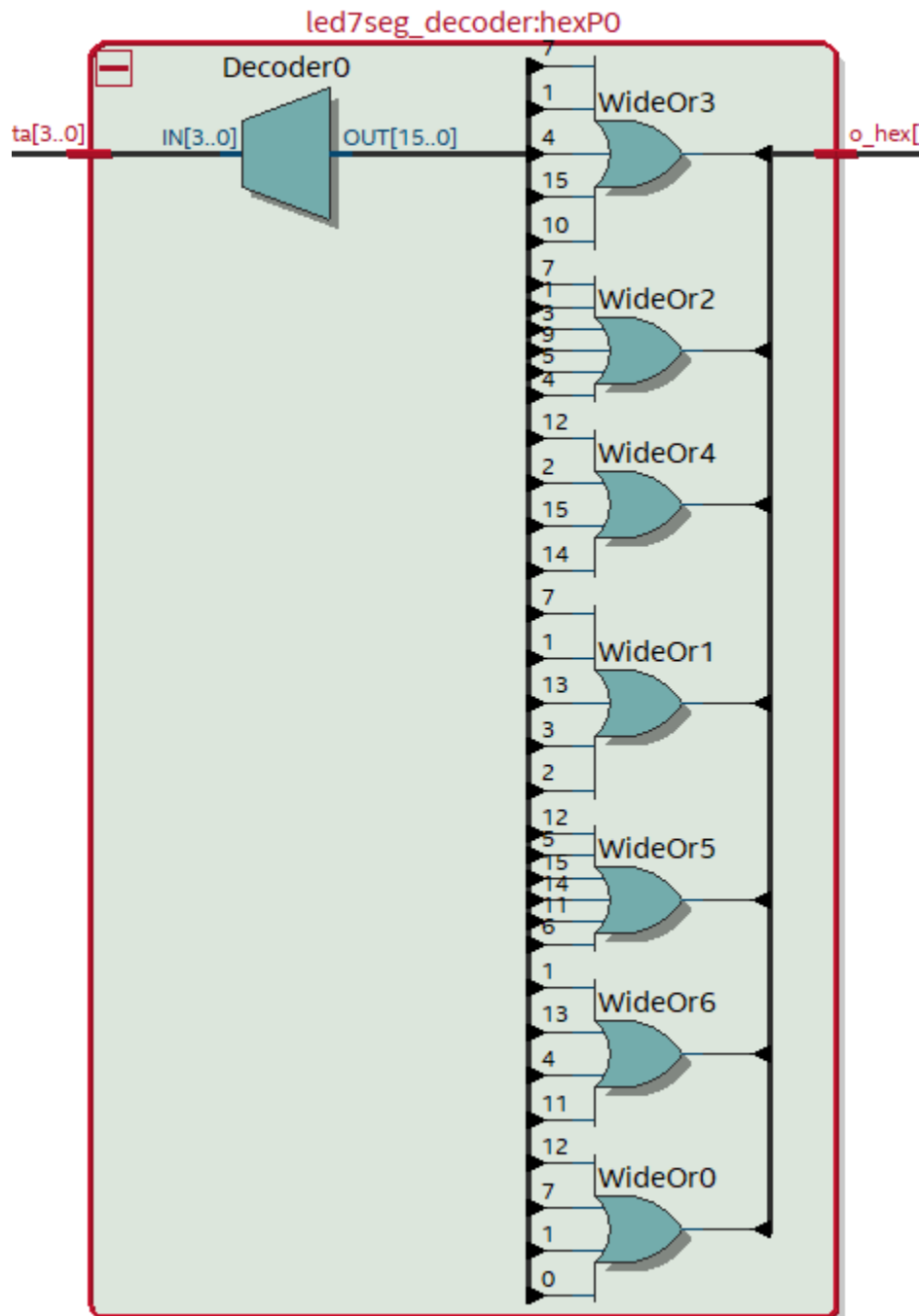
Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Testbench

```
module tb_ex3();
    reg clk;
    reg rst;
    reg En_A;
    reg En_B;
    reg [7:0] A;
    reg [7:0] B;
    wire [15:0] reg_P;

    // Instantiate DUT
    top_mult DUT (
        .clk (clk),
        .rst (rst),
        .En_A (En_A),
        .En_B (En_B),
        .A (A),
        .B (B),
        .reg_P (reg_P)
    );

    // Clock Generation: 10 ns period
    always begin
        #5 clk = ~clk;
    end

    // Stimulus
    initial begin
        // Initialize signals
        clk = 0;
        rst = 1;
        En_A = 0;
        En_B = 0;
        A = 8'b00000000;
        B = 8'b00000000;

        // Apply reset
        #10 rst = 0;
        #10 rst = 1;

        // Test case 1: Multiply 0 * 0
        @(posedge clk); En_A = 1; A = 8'b00000000; En_B = 1; B = 8'b00000000;
        #10;

        // Test case 2: Multiply positive numbers (5 * 3)
        @(posedge clk); En_A = 1; A = 8'b00000101; En_B = 1; B = 8'b00000011;
        #10;
    end
endmodule
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
// Test case 3: Multiply negative numbers (-4 * -2)
@(posedge clk); En_A = 1; A = 8'b11111100; En_B = 1; B = 8'b11111110;
#10;

// Test case 4: Multiply positive and negative (-5 * 6)
@(posedge clk); En_A = 1; A = 8'b11111011; En_B = 1; B = 8'b00000110;
#10;

// Test case 5: Multiply max values (127 * 127)
@(posedge clk); En_A = 1; A = 8'b01111111; En_B = 1; B = 8'b01111111;
#10;

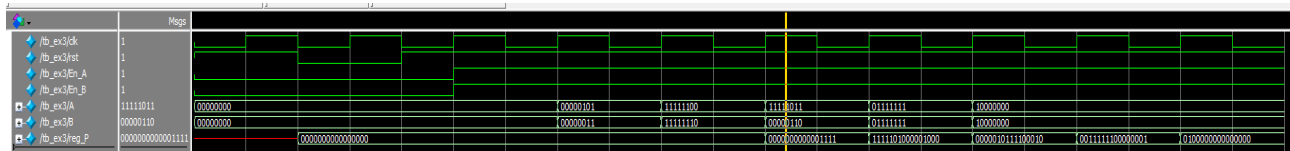
// Test case 6: Multiply min values (-128 * -128)
@(posedge clk); En_A = 1; A = 8'b10000000; En_B = 1; B = 8'b10000000;
#10;

// End of test
#20;
$stop;

end

endmodule
```

Simulation



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Kit

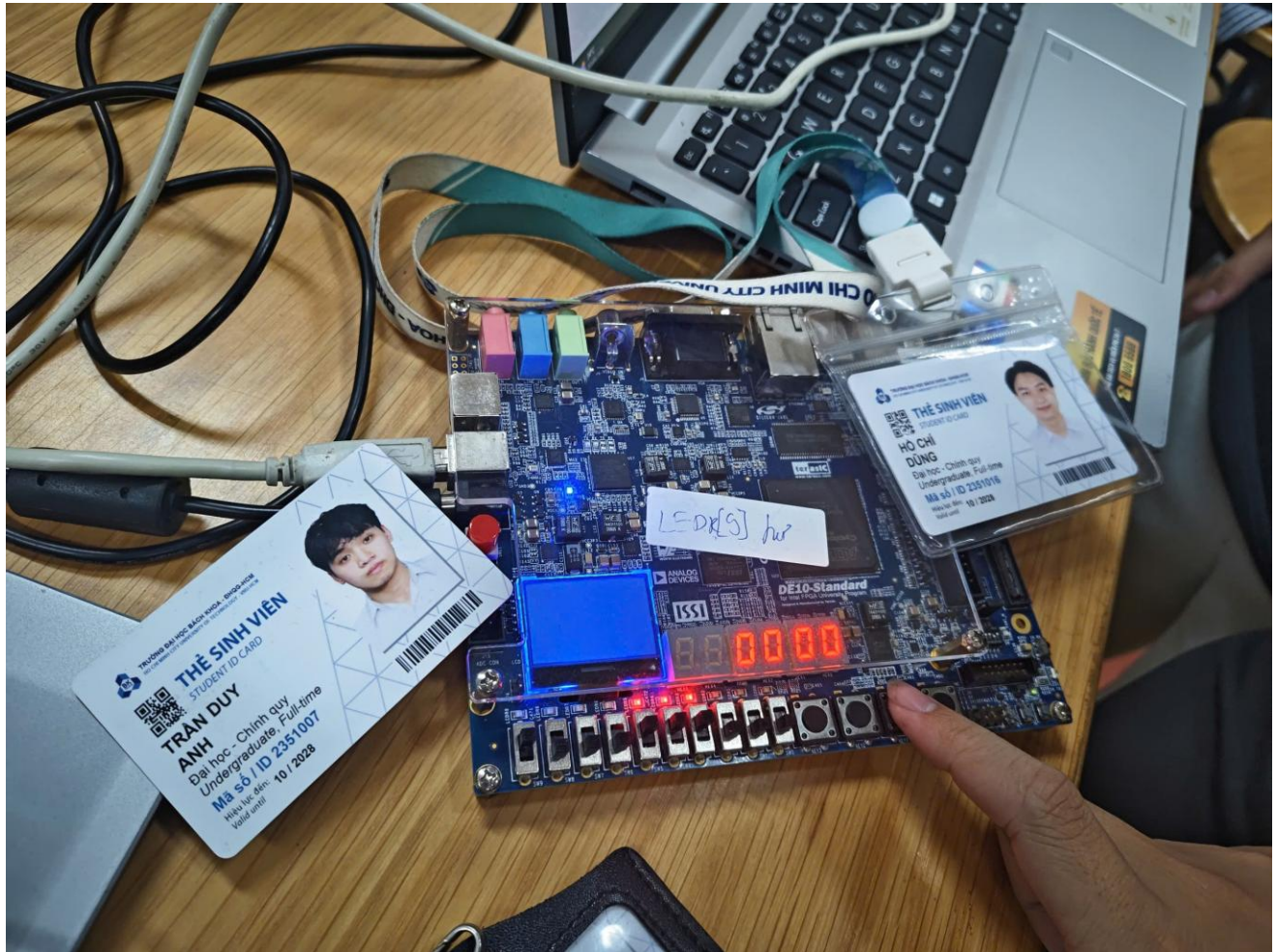


Figure: SW9 = 1, SW8 = 0, Input a = 56



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

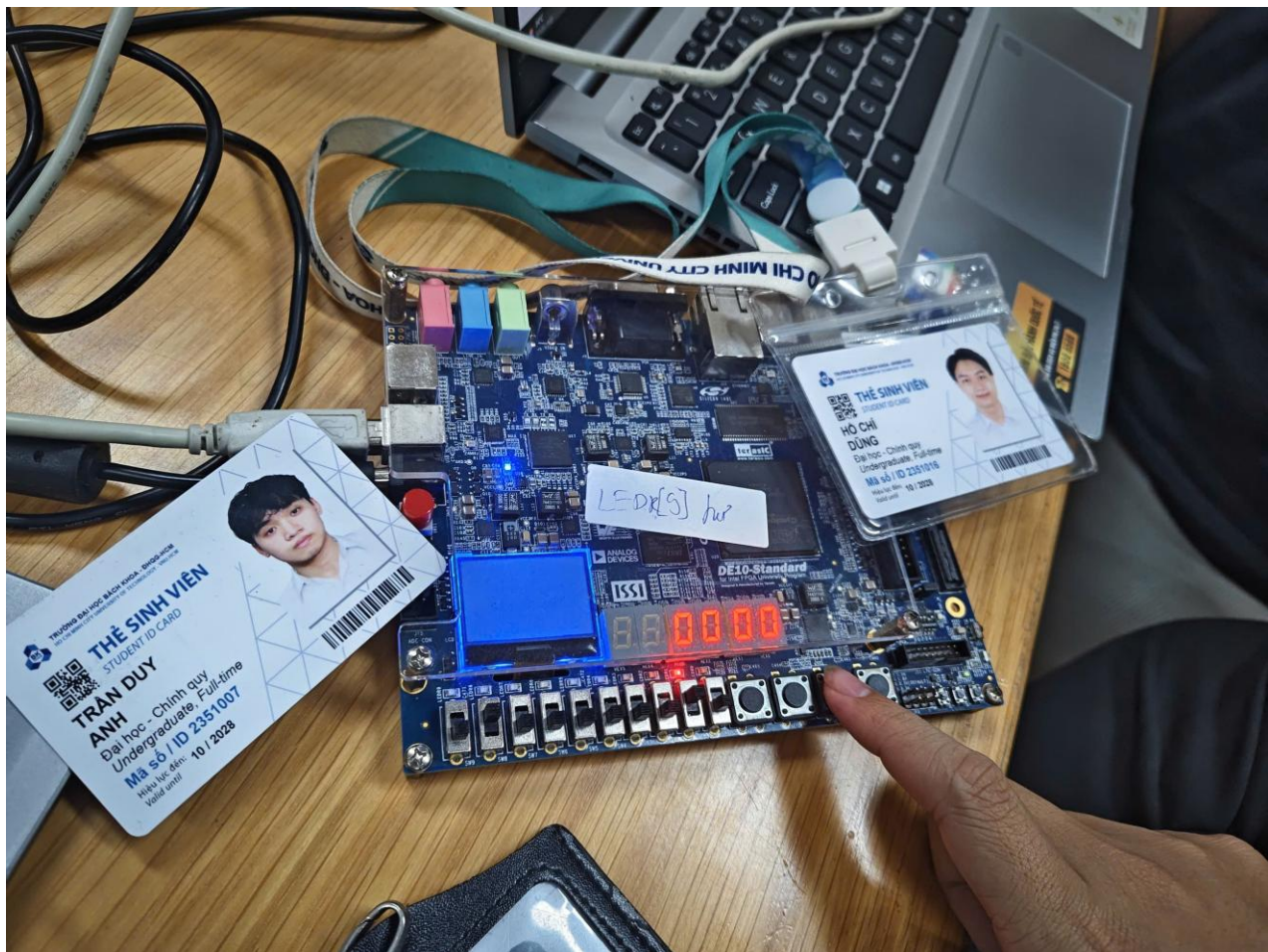


Figure: SW9 = 0, SW8 = 1, Input a = 2

Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

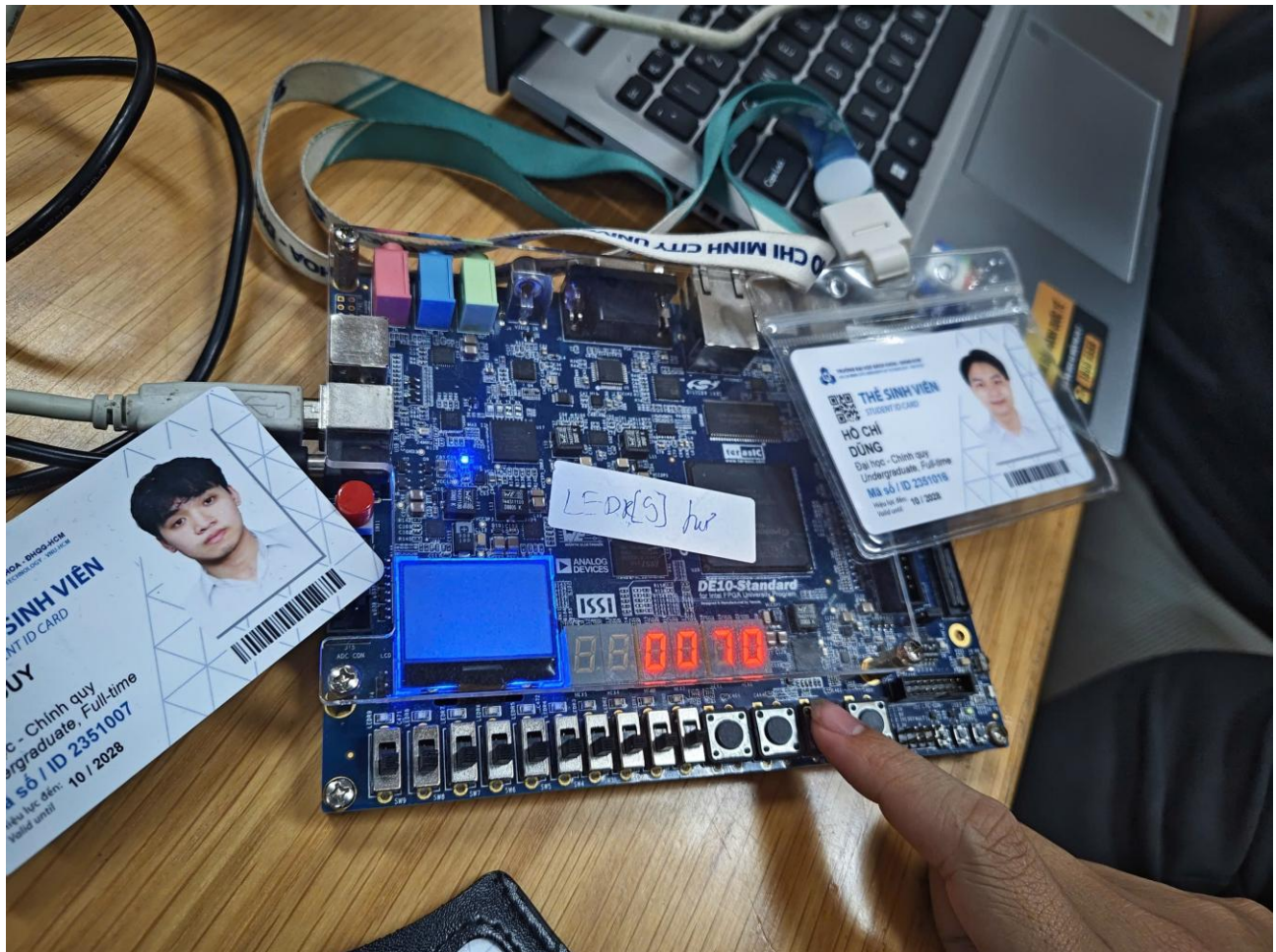


Figure: Display result equal $112_{10} = 70_{16}$

Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

EXPERIMENT 4

Objective: Known how to program ALU circuit with registered inputs and outputs.

Requirement: The circuit in Figure 3 shows an ALU circuit with registered inputs and outputs. The ALU can implement add, subtract and multiply depending on *Sel* input which is shown in Table 1. Write System Verilog code to implement this system.

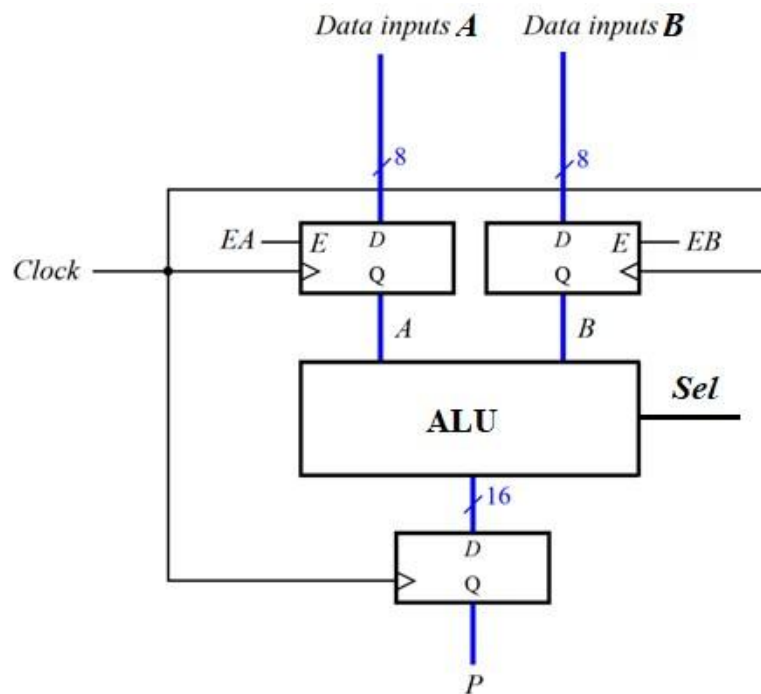


Figure 3: A registered ALU circuit.

<i>Sel</i> Input	ALU operator
000	Add
001	Subtract
010	Multiply

Table 1: Operation of ALU circuit.



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Instruction: This circuit has a lot of inputs (22 inputs) then we can not implement this circuit on DE10 standard. Only test the functionality of your design by inputting various data values and observing the generated outputs.

Check: Your report has to show two results:

- The waveform to prove the circuit works correctly.
- The result of RTL viewer.



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Code

```
module ex4(  
    input logic clk,  
    input logic rst,  
    input logic [2:0] Sel_op,  
    input logic En_A,  
    input logic En_B,  
    input logic [7:0] A,  
    input logic [7:0] B,  
    output logic [7:0] reg_A,  
    output logic [7:0] reg_B,  
    output logic [15:0] P,  
    output logic [15:0] reg_P  
);  
  
eight_bit_register Register_A(  
    .clk (clk),  
    .rst (rst),  
    .D (En_A ? A : reg_A),  
    .Q (reg_A)  
);  
  
eight_bit_register Register_B(  
    .clk (clk),  
    .rst (rst),  
    .D (En_B ? B : reg_B),  
    .Q (reg_B)  
);  
  
alu ALU(  
    .A (reg_A),  
    .B (reg_B),  
    .Sel_op (Sel_op),  
    .P (P)  
);  
  
register_16_bit Register_P(  
    .clk (clk),  
    .rst (rst),  
    .D (P),  
    .Q (reg_P)  
);  
  
endmodule  
module alu(  
    input logic clk,  
    input logic rst,  
    input logic [2:0] Sel_op,  
    input logic En_A,  
    input logic En_B,  
    input logic [7:0] A,  
    input logic [7:0] B,  
    output logic [7:0] reg_A,  
    output logic [7:0] reg_B,  
    output logic [15:0] P,  
    output logic [15:0] reg_P  
);  
  
eight_bit_register Register_A(  
    .clk (clk),  
    .rst (rst),  
    .D (En_A ? A : reg_A),  
    .Q (reg_A)  
);  
  
eight_bit_register Register_B(  
    .clk (clk),  
    .rst (rst),  
    .D (En_B ? B : reg_B),  
    .Q (reg_B)  
);  
  
alu ALU(  
    .A (reg_A),  
    .B (reg_B),  
    .Sel_op (Sel_op),  
    .P (P)  
);  
  
register_16_bit Register_P(  
    .clk (clk),  
    .rst (rst),  
    .D (P),  
    .Q (reg_P)  
);  
  
endmodule
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
    input logic [7:0] A,
    input logic [7:0] B,
    input logic [2:0] Sel_op,
    output logic [15:0] P
);

logic [15:0] p_mult;
logic [7:0] s_add;
logic [7:0] s_subtr;
logic cout_add;
logic cout_subtr;
logic v_add;
logic v_subtr;

always_comb begin
case(Sel_op)
    3'b000:
        if (v_add == 1)    P = {{8{cout_add}}, s_add};
        else                P = {{8{s_add[7]}}, s_add};
    3'b001:
        if (v_subtr == 1)  P = {{8{cout_subtr}}, s_subtr};
        else                P = {{8{s_subtr[7]}}, s_subtr};
    3'b010: P = p_mult;
    default: P = 16'b0;
endcase
end

eight_bit_adder Adder(
    .A (A),
    .B (B),
    .C_in (Sel_op[0]),
    .S (s_add),
    .C_out (cout_add),
    .V (v_add)
);

eight_bit_adder Adder_Subtractor(
    .A (A),
    .B (B),
    .C_in (Sel_op[0]),
    .S (s_subtr),
    .C_out (cout_subtr),
    .V (v_subtr)
);

eight_bit_multiplier Multiplier(
    .A (A),
    .B (B),
    .P (p_mult)
);
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
);

endmodule

module eight_bit_register(
    input logic clk,
    input logic rst,
    input logic [7:0] D,
    output logic [7:0] Q
);

always_ff @( posedge clk or negedge rst ) begin
    if(!rst)
        Q <= 8'b0;
    else
        Q <= D;
end
endmodule

module register_16_bit(
    input logic clk,
    input logic rst,
    input logic [15:0] D,
    output logic [15:0] Q
);

always_ff @( posedge clk or negedge rst ) begin
    if(!rst)
        Q <= 16'b0;
    else
        Q <= D;
end
endmodule

module eight_bit_adder(
    input logic [7:0] A,
    input logic [7:0] B,
    input logic C_in,
    output logic [7:0] S,
    output logic C_out,
    output logic V
);

logic [8:0] carry;

full_adder add0(
    .a (A[0]),
    .b (B[0]^C_in),
    .Cin (carry[0]),
    .s (S[0]),
    .Cout (carry[1])
);
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
full_adder add1(  
    .a (A[1]),  
    .b (B[1]^C_in),  
    .Cin (carry[1]),  
    .s (S[1]),  
    .Cout (carry[2])  
);  
  
full_adder add2(  
    .a (A[2]),  
    .b (B[2]^C_in),  
    .Cin (carry[2]),  
    .s (S[2]),  
    .Cout (carry[3])  
);  
  
full_adder add3(  
    .a (A[3]),  
    .b (B[3]^C_in),  
    .Cin (carry[3]),  
    .s (S[3]),  
    .Cout (carry[4])  
);  
  
full_adder add4(  
    .a (A[4]),  
    .b (B[4]^C_in),  
    .Cin (carry[4]),  
    .s (S[4]),  
    .Cout (carry[5])  
);  
  
full_adder add5(  
    .a (A[5]),  
    .b (B[5]^C_in),  
    .Cin (carry[5]),  
    .s (S[5]),  
    .Cout (carry[6])  
);  
  
full_adder add6(  
    .a (A[6]),  
    .b (B[6]^C_in),  
    .Cin (carry[6]),  
    .s (S[6]),  
    .Cout (carry[7])  
);  
  
full_adder add7(  
    .a (A[7]),  
    .b (B[7]^C_in),  
    .Cin (carry[7]),  
    .s (S[7]),  
    .Cout (carry[8])  
);
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
.a (A[7]),
.b (B[7]^C_in),
.Cin (carry[7]),
.s (S[7]),
.Cout (carry[8])
);

assign carry[0] = C_in;
assign C_out = carry[8];
assign V = carry[8] ^ carry[7];

endmodule
module full_adder(
    input a, b, Cin,
    output s, Cout
);
assign s = a^b^Cin;
assign Cout = a&b | (a^b)&Cin;
endmodule
module eight_bit_multiplier(
    input logic [7:0] A,
    input logic [7:0] B,
    output logic [15:0] P
);

logic [7:0] sub_Sum0;
logic [7:0] sub_Sum1;
logic [7:0] sub_Sum2;
logic [7:0] sub_Sum3;
logic [7:0] sub_Sum4;
logic [7:0] sub_Sum5;
logic [7:0] sub_Sum6;

logic Cout0;
logic Cout1;
logic Cout2;
logic Cout3;
logic Cout4;
logic Cout5;
logic Cout6;

logic V0;
logic V1;
logic V2;
logic V3;
logic V4;
logic V5;
logic V6;
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
eight_bit_adder sub_add0(  
    .A (A[7:1] & {8{B[0]}}),  
    .B (A[7:0] & {8{B[1]}}),  
    .C_in (0),  
    .S (sub_Sum0),  
    .C_out (Cout0),  
    .V (V0)  
);  
assign P[0] = A[0] & B[0];  
assign P[1] = sub_Sum0[0];  
  
eight_bit_adder sub_add1(  
    .A ({Cout0, sub_Sum0[7:1]}),  
    .B (A[7:0] & {8{B[2]}}),  
    .C_in (0),  
    .S (sub_Sum1),  
    .C_out (Cout1),  
    .V (V1)  
);  
assign P[2] = sub_Sum1[0];  
  
eight_bit_adder sub_add2(  
    .A ({Cout1, sub_Sum1[7:1]}),  
    .B (A[7:0] & {8{B[3]}}),  
    .C_in (0),  
    .S (sub_Sum2),  
    .C_out (Cout2),  
    .V (V2)  
);  
assign P[3] = sub_Sum2[0];  
  
eight_bit_adder sub_add3(  
    .A ({Cout2, sub_Sum2[7:1]}),  
    .B (A[7:0] & {8{B[4]}}),  
    .C_in (0),  
    .S (sub_Sum3),  
    .C_out (Cout3),  
    .V (V3)  
);  
assign P[4] = sub_Sum3[0];  
  
eight_bit_adder sub_add4(  
    .A ({Cout3, sub_Sum3[7:1]}),  
    .B (A[7:0] & {8{B[5]}}),  
    .C_in (0),  
    .S (sub_Sum4),  
    .C_out (Cout4),  
    .V (V4)  
);
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
assign P[5] = sub_Sum4[0];

eight_bit_adder sub_add5(
    .A ({Cout4, sub_Sum4[7:1]}),
    .B (A[7:0] & {8{B[6]}}),
    .C_in (0),
    .S (sub_Sum5),
    .C_out (Cout5),
    .V (V5)
);
assign P[6] = sub_Sum5[0];

eight_bit_adder sub_add6(
    .A ({Cout5, sub_Sum5[7:1]}),
    .B (A[7:0] & {8{B[7]}}),
    .C_in (0),
    .S (sub_Sum6),
    .C_out (Cout6),
    .V (V6)
);
assign P[15:7] = {Cout6, sub_Sum6};

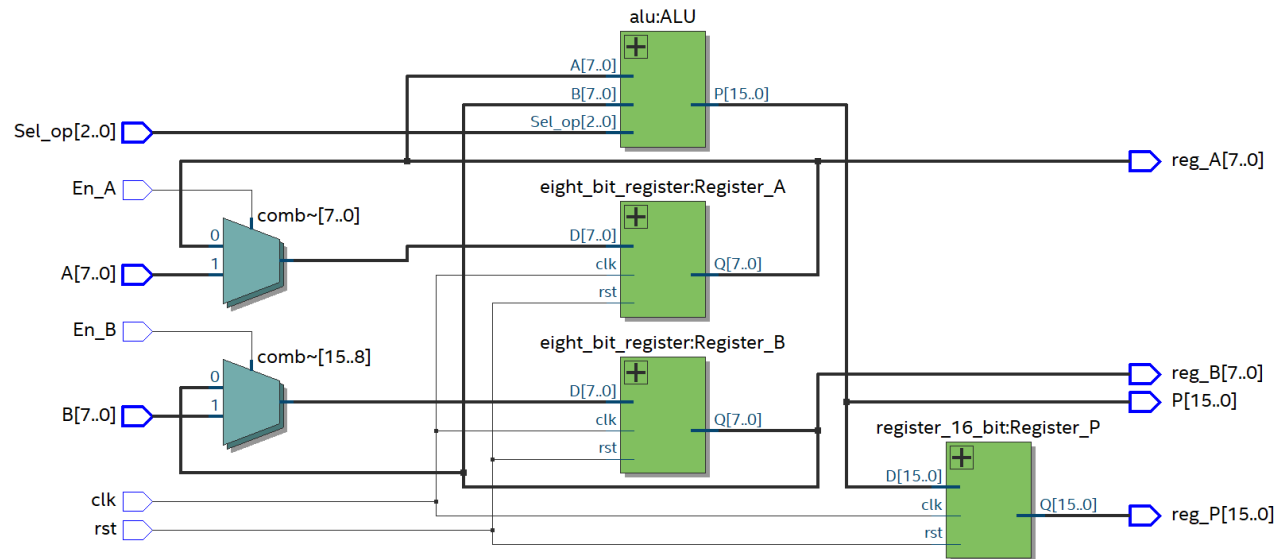
endmodule
```



Laboratory 1:

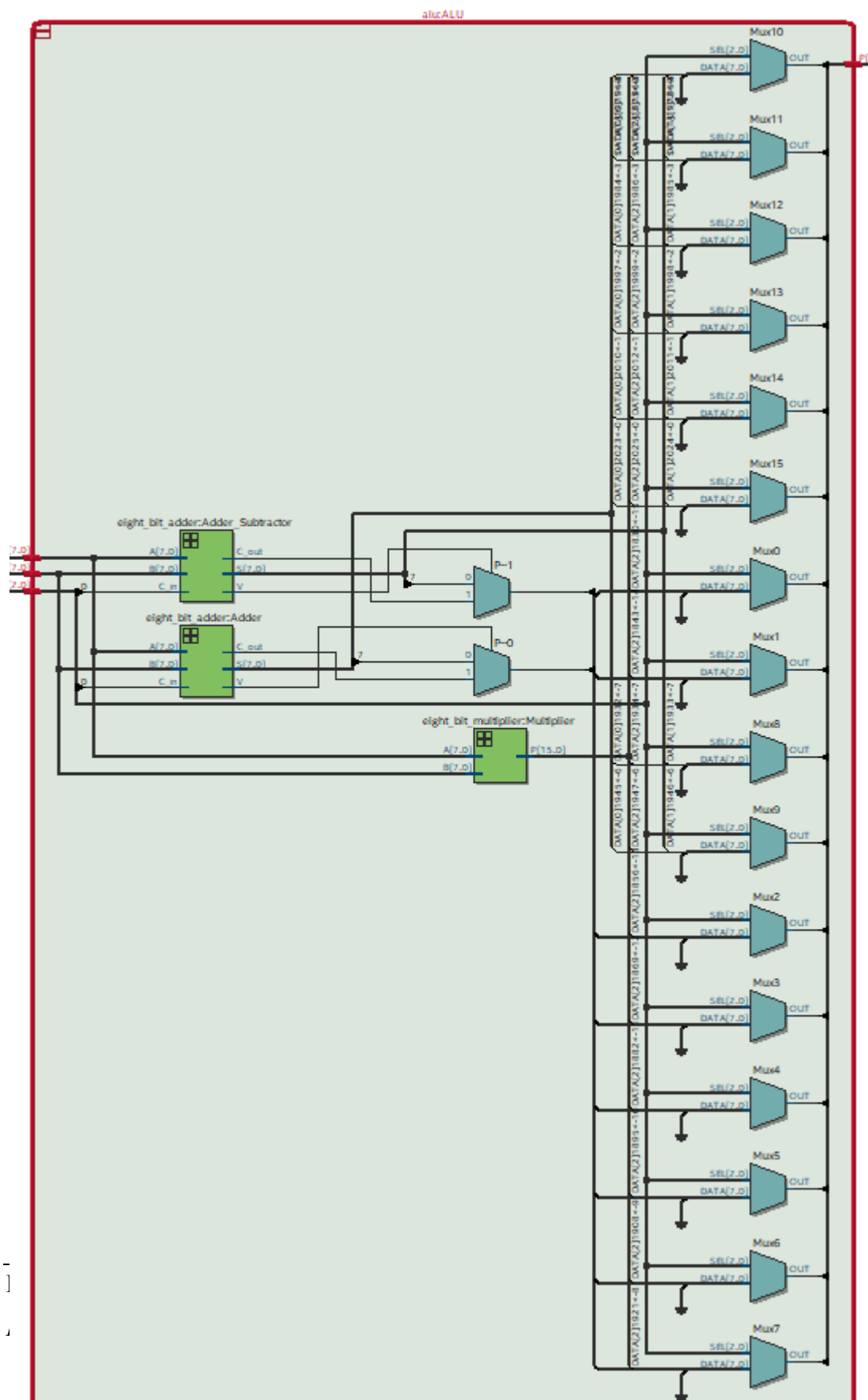
ADDERS, SUBTRACTORS AND MULTIPLIERS

RTL viewer



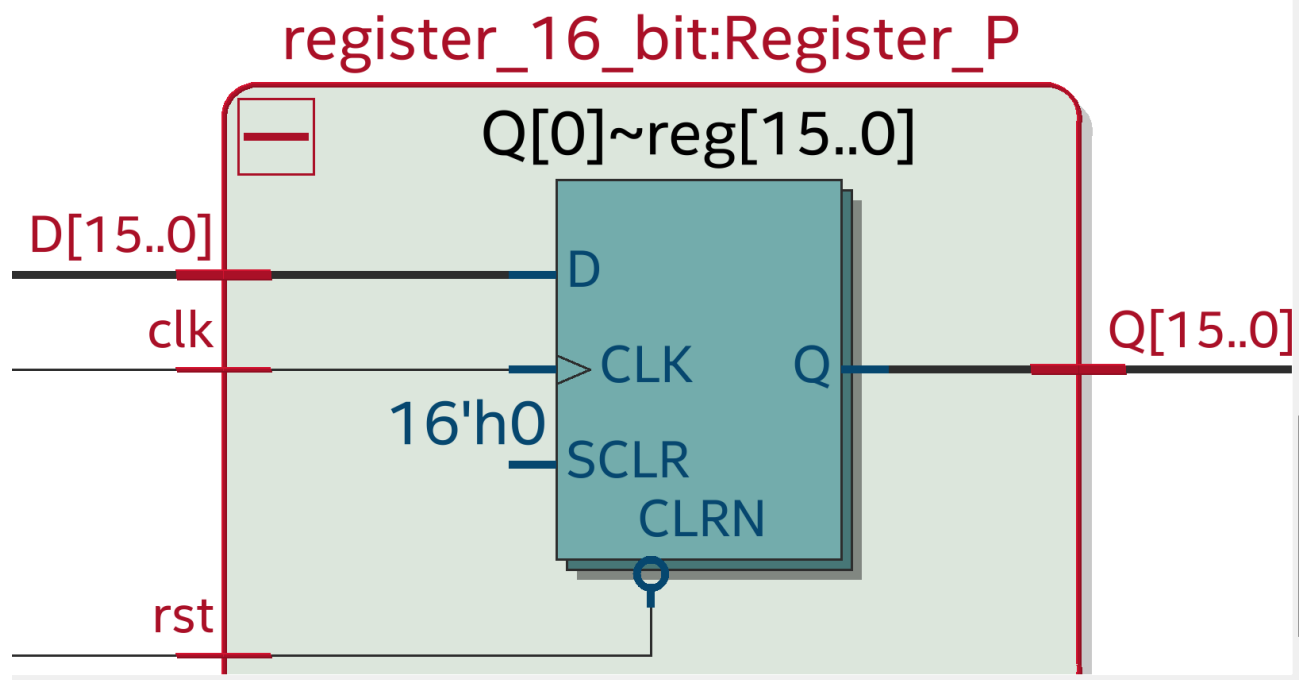
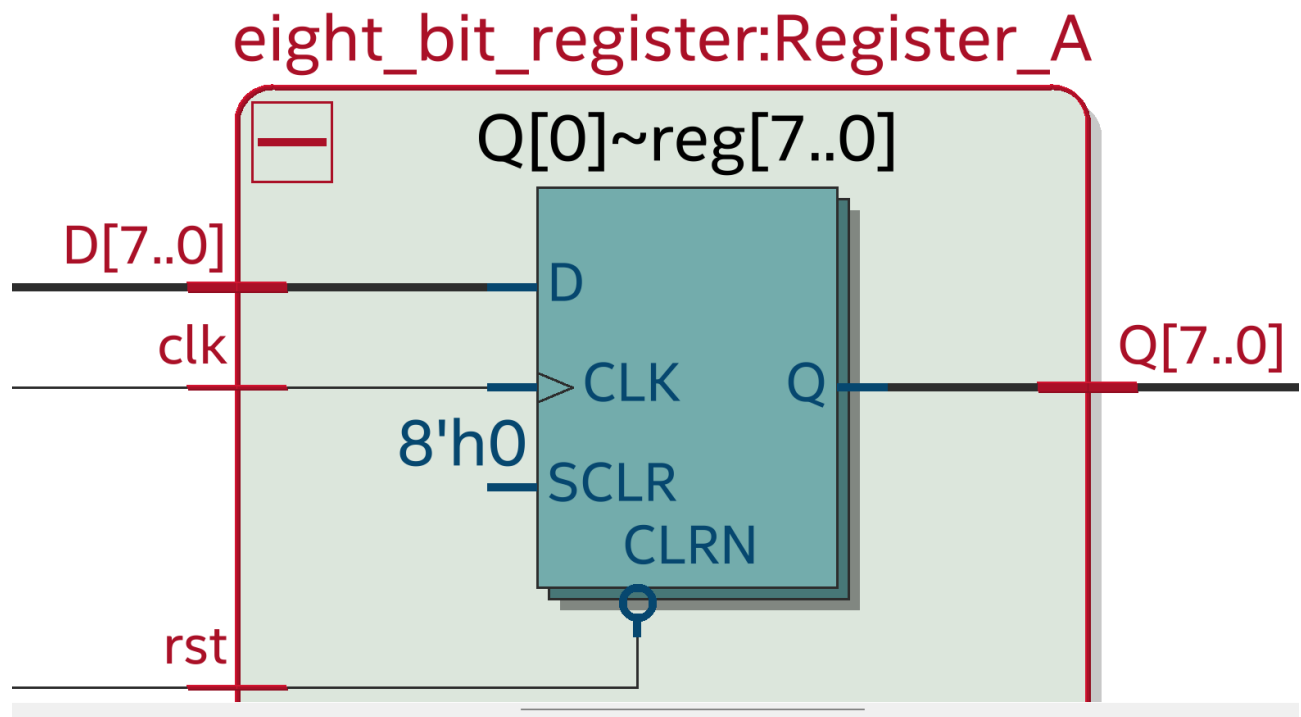
Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Testbench

```
module tb_ex4();
    reg clk;
    reg rst;
    reg [2:0] Sel_op;
    reg En_A;
    reg En_B;
    reg [7:0] A;
    reg [7:0] B;
    wire [7:0] reg_A;
    wire [7:0] reg_B;
    wire [15:0] P;
    wire [15:0] reg_P;

    // Instantiate the DUT
    ex4 DUT (
        .clk (clk),
        .rst (rst),
        .Sel_op (Sel_op),
        .En_A (En_A),
        .En_B (En_B),
        .A (A),
        .B (B),
        .reg_A (reg_A),
        .reg_B (reg_B),
        .P (P),
        .reg_P (reg_P)
    );

    // Clock generation (10ns period -> 5ns high, 5ns low)
    always begin
        #5 clk = ~clk;
    end

    // Testbench procedure
    initial begin
        // Initialize signals
        clk = 0;
        rst = 1;
        En_A = 0;
        En_B = 0;
        A = 0;
        B = 0;
        Sel_op = 3'b000;

        // Reset sequence
        #10 rst = 0;
    end
endmodule
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

```
#10 rst = 1;
#10;

// Test Addition (5 + 10)
@(posedge clk); En_A = 1; A = 8'd5;
@(posedge clk); En_B = 1; B = 8'd10;
@(posedge clk); Sel_op = 3'b000; En_A = 0; En_B = 0;
#20;

// Test Subtraction (20 - 15)
@(posedge clk); En_A = 1; A = 8'd20;
@(posedge clk); En_B = 1; B = 8'd15;
@(posedge clk); Sel_op = 3'b001; En_A = 0; En_B = 0;
#20;

// Test Multiplication (8 * 4)
@(posedge clk); En_A = 1; A = 8'd8;
@(posedge clk); En_B = 1; B = 8'd4;
@(posedge clk); Sel_op = 3'b010; En_A = 0; En_B = 0;
#20;

// Test Edge Case: Overflow in Addition (127 + 1)
@(posedge clk); En_A = 1; A = 8'd127;
@(posedge clk); En_B = 1; B = 8'd1;
@(posedge clk); Sel_op = 3'b000; En_A = 0; En_B = 0;
#20;

// Test Edge Case: Underflow in Subtraction (-128 - 1)
@(posedge clk); En_A = 1; A = -8'd128;
@(posedge clk); En_B = 1; B = 8'd1;
@(posedge clk); Sel_op = 3'b001; En_A = 0; En_B = 0;
#20;

// Test Large Multiplication (50 * 50)
@(posedge clk); En_A = 1; A = 8'd50;
@(posedge clk); En_B = 1; B = 8'd50;
@(posedge clk); Sel_op = 3'b010; En_A = 0; En_B = 0;
#20;

// End of test
$stop;
end
endmodule
```



Laboratory 1:

ADDERS, SUBTRACTORS AND MULTIPLIERS

Simulation

